

C-Shell Project Architectural Guide

<https://gemini.google.com/app/d5f2dd21cc678030>

User prompt: Act as an expert Operating Systems and C programming mentor. I am working on a custom C-Shell project for my OS course. I want to deeply understand how to architect and build this project myself, so I am strictly looking for architectural guidance, data structure design, and logical breakdowns. ****CRITICAL INSTRUCTION:**** Do NOT generate complete, copy-pasteable C code for any part of this project. Your goal is to guide my understanding, not do the work for me. If you need to explain logic, use high-level pseudocode, diagrams, or mention the specific POSIX system calls (like 'fork', 'execvp', 'dup2', 'pipe', 'opendir', 'stat', 'waitpid') I need to research and use. Please read the assignment instructions I have pasted below and provide: 1. ****High-Level Architecture:**** How should I modularize my codebase into different '.c' and '.h' files? 2. ****Data Structures:**** What structs will I need to design (e.g., for parsing tokens, tracking commands, and tracking background jobs)? 3. ****Component Breakdown:**** For each major part of the specification (e.g., Prompt, Built-ins, Redirections, Pipelines, Background Execution), explain the step-by-step logical flow. 4. ****Edge Cases & Pitfalls:**** What tricky edge cases (such as memory leaks, zombie processes, blocked pipes, or string truncation) should I watch out for during implementation? First list all the syscalls i will need in a separate list, also do this only for part A,B and C, skip everything after that Here are the project instructions: CS3.301 Operating Systems and Networks CS3.301 Operating Systems and Networks HomeCalendarLectures Introduction to OSProcess VirtualizationProcess APILimited Direct ExecutionScheduling Mini-Projects Mini Project 1 Tutorials System Calls & Intro to xv6 Homework General InstructionsProcesses & Syscalls Course ProjectResources TextbooksMan Pagesxv6Git Course PolicyStaff Mini-Projects Mini Project 1 Mini-Project 1 · Released Aug 14, 2026 · Due Sep 13, 2026 Implement your very own C-Shell & Implement the MLFQ scheduling policy for xv6. Before you start This is an individual assignment. Deadlines There are two submissions for this assignment. You are expected to submit parts A to C of C-Shell for the mid-submission, and remainder of the assignment for the end-submission. Mid-submission: August 24, 11:59 pm End-submission: September 13, 11:59 pm You may use your late days for both of the submissions. Submission The exact details about submission will be made clear shortly. However, you NEED to maintain a git repository for the duration of the project and commit iteratively as you go. Making a bunch of big commits right before the deadline may result in penalization. And before you ask: you DO NOT need a GitHub repo to maintain a git repo. You can initialize one locally and keep it there until we figure out submission. You can refer to the resource on git for more details. Make sure your repository has the following structure: mini-project1/ ├── c-shell/ | ├── src/ | ├── include/ | └── Makefile ├── xv6/ | ├── user/ | ├── mkfs/ | ├── kernel/ | ├── Makefile | └── report.md # Or report.pdf └── README.md Grading You will be graded based on both your implementation and an in-person evaluation with a TA. You are expected to understand the code that you submit. Doubt Doc Doubts Answers All the best! Have fun :) C Shell [200] General Requirements Break the project down into multiple .c and .h files based on functionality. Monolithic code will be penalized. Use only C POSIX library headers and functions. You can use this for reference. Compile with the following flags for POSIX compliance: gcc -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 -Wall -Wextra -Werror -Wno-unused-parameter -fno-asn \ your_file.c Final build must compile with make all in the shell directory, producing shell.out in that same directory. References These could be helpful for you to understand things required to implement this section. POSIX.1-2017 spec man 2 pages for fork, exec, wait, pipe, dup2, open. These cover most of what you'll need. Use the source of actual bash (will be helpful to understand how to do certain parts of the mini-project) Part A: Shell Input [20] This is the base for the rest of your project. Banned syscalls for Part A: exec* (any syscall that starts with exec) A1: Shell Prompt [3] Your shell should show a prompt so the user knows it's ready for input. <username@hostname:currentpath> Requirements: Display the prompt whenever the shell isn't running a foreground process. The directory the shell is started in becomes its home directory. If the current working directory has the home directory as an ancestor (e.g. /path/to/home/osn), replace the home directory prefix with ~. So the example becomes ~/osn. If the current working directory does not have the home directory as an ancestor, show the absolute path as is. Example <goatam@iiit:~/osnmp1/> make<goatam@iiit:~/osnmp1/> ./shell.out<goatam@iiit:~/osnmp1/> A2: User Input [2] Requirements: The shell should allow a user to type input. On enter/return, the shell should consume the input. After consuming input, the shell should display the prompt again. Example <goatam@iiit:~/osnmp1/> Hi there guys!<goatam@iiit:~/osnmp1/> This shell is cool!<goatam@iiit:~/osnmp1/> A3: Input Parsing [15] Your shell must scan and validate every line before executing anything. This section defines two layers: a lexer that turns characters into tokens, and a grammar that validates the token sequence. Validation must be performed. Note: Since both Lexer and Parser follow a regular grammar, use your Automata Theory skills to the test. Lexer: The lexer reads the raw input line and produces a sequence of tokens, some specifications are given below. Whitespace (space, tab, newline, carriage return) separates tokens and is otherwise discarded - it is never itself a token. Character Classes: special -> | & > < ; quote -> " | ' escape -> \space -> ' | \t | \n | \ordinary -> any character that is none of the above Tokens: OP_PIPE -> | OP_AMP -> & OP_SEMI -> ; OP_LT -> < OP_GT -> > OP_GTGT -> >> WORD -> fragment+fragment -> ordinary | escape any_char | " dq_body " | ' sq_body 'dq_body -> (escape any_char | any char except " and \) *sq_body -> (any char except ') * Apply maximal munch: i.e. at each position, take the longest match. e.g. >> is therefore one token, never two >. Some details on Quoting: ConstructEffectOn the unquoted value \c outside quotesContributes c literally; the backslash is removed"...Contributes the body; \" becomes ", \\ becomes \; any other \c stays as both characters'...'Contributes the body verbatim; no escape processing, so \ is literalSpecial characters inside quotes or escapedLose their operator meaning entirely Lexical errors: The lexer must reject and print cshell: invalid syntax when: A quote is opened and never closed before end of line. A \ appears as the final character of the line. Note: There is no line continuation and no multi-line input. Hint: use EOF to your advantage. Grammar: Terminals are the token classes above. Start symbol is LINE LINE -> ε | WORD ARGARG -> ε | WORD ARG | OP_LT TGT | OP_GT TGT | OP_GTGT TGT | OP_PIPE CMD | OP_SEMI CMD | OP_AMP BGCMD -> WORD ARGTGT -> WORD ARGBG -> ε | WORD ARG Note: This is a right linear grammar, enjoy. TGT is the name of a file or a redirection output, CMD is a valid command Instructions: You do not need a full AST, any structure you find convenient (like a Linked List) works. Just make sure you are representing enough information internally to allow further sections to be implemented. You can use any type of parser and lexer, as long as it conforms to the grammar provided. Requirements Verify whether an inputted command is valid or invalid per the grammar. If invalid, print cshell: invalid syntax. A line that is empty or contains only whitespace is valid: print a fresh prompt. Whitespace between tokens is ignored, in any amount. Whitespace inside quotes is preserved. Examples: <goatam@iiit:~/osnmp1/> cat meow.txt | meow; meow > meow.txt<goatam@iiit:~/osnmp1/> echo a & echo b &<goatam@iiit:~/osnmp1/> cat < a.txt < b.txt > out.txt<goatam@iiit:~/osnmp1/> echo "line one" > a.txt<goatam@iiit:~/osnmp1/> echo "a > b"<goatam@iiit:~/osnmp1/> cat my\ file.txt<goatam@iiit:~/osnmp1/> echo ""<goatam@iiit:~/osnmp1/> reveal -ta -aaa -tt<goatam@iiit:~/osnmp1/>

`cat meow.txt | ; meowshell: invalid syntax<goatam@iiiit::~/~osnmp1> echo hi && cshshell: invalid syntax<goatam@iiiit::~/~osnmp1> cat <cshell: invalid syntax<goatam@iiiit::~/~osnmp1> cat << fcshell: invalid syntax<goatam@iiiit::~/~osnmp1> | sortcshell: invalid syntax<goatam@iiiit::~/~osnmp1> echo "unclosedcshell: invalid syntax<goatam@iiiit::~/~osnmp1> echo trailing\cshell: invalid syntax<goatam@iiiit::~/~osnmp1>`

Part B: Shell Intrinsics [40] These are commands any shell worth its salt supports. Banned syscalls for Part B: `exec*` (any syscall that starts with `exec`) B1: `hop` [10] Syntax: `hop ((~ | . | .. | - | name)*)?` Purpose: The `hop` command changes the shell's current working directory. Unlike a plain `cd`, `hop` falls back to a frequency lookup when a name doesn't resolve directly, so you can jump to directories you visit often without typing the full path. Requirements Execute the following sequentially for each passed argument: "~" or no arguments: change the CWD to the shell's home directory. "." : do nothing, stay in the same CWD. "..": change the CWD to the parent directory of the CWD, or do nothing if the CWD has no parent. "-": change the CWD to the previous CWD, or do nothing if there was no previous CWD. "name": if this resolves to a valid relative or absolute path from the CWD, change to it directly. If "name" does not resolve directly, fall back to a frequency lookup as defined below. If neither a direct match nor a frequency match is found, output `hop: no such directory` Frequency Keep a persistent record of directories hopped into, across shell sessions. Every successful `hop` updates that directory's standing. Standing depends on both frequency and recency of visits. When "name" doesn't resolve directly, `hop` into the highest ranked directory whose path contains "name" as a substring. If the top match no longer exists on disk, skip it and try the next best. Scoring, decay, and storage are up to you, but must be deterministic. Resource: same idea behind `zoxide's cd` replacement: <https://github.com/ajeetsouza/zoxide/wiki/Algorithm> Example `<goatam@iiiit::~/~osnmp1> hop ~<goatam@iiiit::~/~osnmp1> hop ..<goatam@iiiit::~/home/> hop goatam/osnmp1 ..<goatam@iiiit::~/~osnmp1> hop osn<goatam@iiiit::~/~osnmp1>` B2: `reveal` [10] Syntax: `reveal (-a|t)* (~ | . | .. | - | name)?` Purpose: The `reveal` command lists the files and directories in a directory. Requirements If no argument is passed, `reveal` lists the contents of the current working directory. "a": reveal all files and directories, including hidden ones (starting with `.`). Default is to hide them. "t": recursively reveal the contents of all subdirectories. the content of the subdirectory must be displayed after the directory itself (refer example below) When both flags are set, recursively show all files, hidden included, in the format given below. When neither flag is set, print entries in the format of `ls`. The argument passed resolves the same way as `hop's ~, ., .., -, and direct path cases`, except `reveal` lists directory contents instead of changing the CWD. There is no frequency fallback: if the argument does not resolve to an existing directory, print `reveal: no such directory`. Entries are always listed in lexicographic order, sorted by ASCII value. If `reveal` is passed too many arguments, print: `reveal: invalid syntax` If `reveal` is passed an invalid flag, print: `reveal: invalid syntax` If `reveal -l` is run before any `hop` has been run since shell start, print: `reveal: no such directory` Example `<goatam@iiiit::~/~osnmp1> reveal ~<goatam@iiiit::~/~osnmp1> hop ..<goatam@iiiit::~/home> revealgoatam<goatam@iiiit::~/home> hop goatam/osnmp1<goatam@iiiit::~/~osnmp1> reveal -tMakefileREADME.mdinclude/include/parser.hinclude/shell.hshell.outsrc/src/main.csrc/parser.c<goatam@iiiit::~/~osnmp1> reveal -taMakefileREADME.mdinclude/include/parser.hinclude/shell.hshell.outsrc/src/main.csrc/parser.c<goatam@iiiit::~/~osnmp1> reveal -ta -ttttttaaaaaaaaaaaatttt -aaaa -tttMakefileREADME.mdinclude/include/parser.hinclude/shell.hshell.outsrc/src/main.csrc/parser.c` B3: `peek` [10] Syntax: `peek (-n|r)* filename*` Purpose: The `peek` command concatenates the contents of one or more given files (or standard input) and then prints it to the standard output. Requirements: "n": counts the number of non-empty lines in the input. The line number must be printed before the corresponding line. "r": print the contents of the input in reverse line order. For seekable input (regular files), use `lseek` to read the file backwards in fixed-size chunks rather than loading it all into memory at once. For non-seekable input (pipes, standard input), buffering the full input before reversing is allowed. When both flags are set, print the contents of the file in reverse order, with its corresponding line number. When no flag is set, print the contents of the file as is. If more than one file is provided, concatenate their contents in the given order. When `-r` is combined with multiple files, each file's lines are reversed independently before concatenation; the overall file order given on the command line is preserved. If no filename is provided, or when filename is `-`, read from standard input. If filename does not exist, print `peek: no such file or directory` If filename is a directory, print `peek: is a directory` Example `<goatam@iiiit::~/~osnmp1> peek README.md# OSN MP1This is OSN shell assignment.<goatam@iiiit::~/~osnmp1> peek -n README.md1 #OSN MP12 This is OSN shell assignment<goatam@iiiit::~/~osnmp1> peek -r README.md2 This is OSN shell assignment# OSN MP1<goatam@iiiit::~/~osnmp1> peek -rn -rrrrnnnnn -nrnnrr README.md2 This is OSN shell assignment1 #OSN MP1<goatam@iiiit::~/~osnmp1> peek -meowmeow<goatam@iiiit::~/~osnmp1> peek README.md Makefile#OSN MP1This is OSN shell assignment.CC = gccCSTD = -std=c99Preprocessor definitionsDEFINES = -D_POSIX_C_SOURCE=200809L-D_XOPEN_SOURCE=700` B4: `locate` [10] Syntax: `locate filename*` Purpose: The `locate` command returns the pathnames of the files (or links) which would be executed in the current environment, had its arguments been given as commands in a strictly POSIX-conformant shell. Requirements: For each filename, first check the current working directory. If a match exists there, print its absolute path first. Then search each directory listed in `PATH`, in order, and print the absolute path of every match found. This means a name matching in both the cwd and two `PATH` entries prints three lines, in that order. The absolute path to the executable must be printed (even if the executable exists in the current working directory). If multiple arguments are provided, print the path for each one of them, in the order specified by the user. If multiple matching executable paths are found, print all the paths. If filename is not a valid executable, print: `locate: command not found (commandname)` and continue processing the remaining arguments. You may assume that file paths will not be passed as arguments. Example: `<goatam@iiiit::~/~osnmp1> locate java/usr/bin/java<goatam@iiiit::~/~osnmp1> locate man java python nada/usr/bin/man/usr/bin/java/usr/bin/pythonlocate: command not found (nada)<goatam@iiiit::~/~osnmp1> locate sudo/usr/bin/sudo/bin/sudo` Part C: File Redirection and Pipes [50] For this part, when processing commands with sequential (`;`) or background (`&`) operators, you should only execute the first command group and ignore the rest. C1: Command Execution [8] You must allow execution of arbitrary commands, found either as a relative/absolute path to an executable, or by name. Requirements If the command name contains a `/`, treat it as a literal path and execute it directly if it exists and is executable. If the command name does not contain a `/`, first check whether an executable with that name exists in the current working directory, and run it if so. If no such executable exists in the current working directory, search `PATH` as usual. To skip the current directory check and search `PATH` directly, prefix the command name with `%`. %name must always resolve via `PATH`, even if a matching executable exists in the current directory. If no executable is found by any of the above, output `cshell: command not found (commandname)`. Example `<goatam@iiiit::~/~proj> echo hellohello<goatam@iiiit::~/~proj> lsbuild.sh<goatam@iiiit::~/~proj> build.shrunning local build script<goatam@iiiit::~/~proj> %build.shcshell: command not found (build.sh)<goatam@iiiit::~/~proj> dosakdaoskdoscshell: command not found (dosakdaoskdos)` C2: Input Redirection [12] Syntax: `command < filename` Purpose: The input redirection operator allows a command to read its standard input from one or more files instead of the terminal. Requirements The shell must open each specified file for reading using the `open()` system call with `O_RDONLY` flag. If any file does not exist or cannot be opened, the shell must print `cshell: no such file or directory` and not execute the command. A command may have more than one input redirection (e.g. `command < f1 < f2`). The command's standard

input must consist of the contents of every listed file, read in the order given, as one continuous stream. The shell must redirect the command's standard input (STDIN_FILENO) to this stream using dup2(). The shell must close any file descriptors it opens once they are no longer needed.

Example `<goatam@iit:~> cat < notes.txtmeeting at 5pm<goatam@iit:~> echo "line one" > a.txt<goatam@iit:~> echo "line two" > b.txt<goatam@iit:~> cat < a.txt < b.txtline oneline two<goatam@iit:~> cat < missing.txtcshell: no such file or directory C3: Output Redirection [12] Syntax: command > filename or command >> filename Purpose: The output redirection operators allow a command's output to be written to one or more files instead of the terminal. Requirements A command may have more than one output redirection (e.g. command > f1 >> f2 > f3). Every listed file must receive the complete output of the command. Each redirection keeps its own mode independently: > truncates its file (or creates it with permissions 0644 if it doesn't exist), >> appends to its file (or creates it with 0644 if it doesn't exist). When output redirection is present, the command's output must not appear on the terminal, only in the listed files. If any listed file cannot be opened for writing, the shell must print cshell: unable to create file for writing and not execute the command. Input and output redirection must work together (e.g. command < input.txt > output.txt >> log.txt). Example <goatam@iit:~> echo hi > out1.txt > out2.txt<goatam@iit:~> cat out1.txt<goatam@iit:~> cat out2.txt<goatam@iit:~> echo again >> out1.txt > out3.txt<goatam@iit:~> cat out1.txt<goatam@iit:~> cat out3.txtagain C4: Command Piping [18] Syntax: command1 | command2 | ... | commandN Purpose: The pipe operator connects one command's output to the next command's input. Requirements The shell must create pipes using pipe() for each | in the command. For each command in the pipeline, the shell must fork a child process. The shell must redirect just the standard output of command[i] to the write end of pipe[i]. The shell must redirect the standard input of command[i+1] to the read end of pipe[i]. The parent shell must close every pipe file descriptor it holds after forking. Each child must close any pipe file descriptors it does not use before executing its command. The parent shell must wait for all commands in the pipeline to complete before continuing. If a command in the pipeline fails to execute, the shell prints cshell: command not found (commandname) for that stage, and the rest of the pipeline still runs. File redirection and pipes must work together (e.g. command1 < input.txt | command2 > output.txt). Example <goatam@iit:~> printf "banana\napple" | sortapplebanana<goatam@iit:~> badcmd | sortcshell: command not found (badcmd)<goatam@iit:~> cat < in.txt | sort > out.txt Part D: Sequential and Background Execution [25] D1: Sequential Execution [10] Syntax: command1 ; command2 ; ... ; commandN Purpose: The semicolon operator allows an arbitrary number of commands to execute one after another. Requirements: The shell must execute each command in the order they appear. The shell must wait for each command to complete before starting the next. If a command fails to execute, the shell must print cshell: command not found (commandname) and then stop executing the remaining commands in sequence. Each command in the sequence must be treated as a complete shell_cmd as defined in the grammar. The shell prompt must be displayed either after all the commands completed successfully, or because one command has failed. <goatam@iit:~> echo hello ; echo worldhelloworld<goatam@iit:~> echo hello ; nada ; echo worldhellocshell: command not found (nada) D2: Background Execution [15] Syntax: command1 & command2 & ... & commandN & Purpose: The ampersand operator allows the command preceding it to run in the background while the shell continues to accept new commands or execute the remaining commands. Requirements: When a command ends with &, the shell must fork a child process and execute it in the background without waiting for it to complete. When multiple commands are separated by &, each command must be launched independently in the order in which it appears. The shell must print the background job number and process ID in the format: [job_number] process_id Job numbers are assigned session-wide and increase monotonically. They are never reused, even after a job completes. The shell must immediately print a new prompt after launching all the background processes. The shell must install a SIGCHLD handler to detect when a background process completes. The SIGCHLD handler must reap terminated processes using waitpid() with WNOHANG, so that the shell never blocks while handling background processes. When a background process completes successfully, the shell must print: command with pid process_id exited normally. When a background process exits abnormally, the shell must print: command with pid process_id exited abnormally. As soon as a background process terminates, the shell must report its completion, including when waiting for user input. If any foreground process is being executed while a background process terminates, its completion must be reported after the foreground process terminates. Background processes must not have access to the terminal for input. For a pipeline, the reported pid should be the pid of the first command in the pipeline. Note: You may have to do the first part of E1 to be able to implement functionality #12 of this section. Example <goatam@iit:~> echo hello & sleep 10 &hello[1] 4021[2] 4022echo with pid 4021 exited normally<goatam@iit:~>sleep with pid 4022 exited normally<goatam@iit:~> sleep 1 & sleep 2 & cat[1] 4023[2] 4024hellohello^Csleep with pid 4023 exited normally Part E: Exotic Shell Intrinsics [45] E1: activities [10] Syntax: activities Purpose: The activities command lists every process the shell has spawned that is still running, grouped by process group. Requirements Every pipeline the shell launches must run in its own process group, using setpgid() on each child right after fork() in the parent, and also in every child before execve. A standalone command (no |) counts as a process group of one. activities must print one line per process, followed by one line per process in that group, indented under it. Each process line must show: pid, command name, and state (Running or Stopped). Print format: Group line: [job_number] pgid pgid_value Process line: pid command_name state (two-space indent, fields space-separated; exact column alignment does not matter) Groups must be printed in the order they were launched, oldest first. Processes that have exited must be removed from the list before printing. For a pipeline, consider the pid of the first command in the pipeline. Example <goatam@iit:~> sleep 100 &[1] 4021<goatam@iit:~> cat | sort &[2] 4030<goatam@iit:~> activities[1] pgid 4021 4021 sleep Running[2] pgid 4030 4030 cat Running 4031 sort Running E2: Terminal Control [15] Purpose: Ctrl-C, Ctrl-Z, and Ctrl-D let the user interrupt, stop, or end shell sessions the way any real shell supports. Requirements The shell must ignore SIGINT and SIGTSTP and SIGTTOU. Before running a foreground pipeline, the shell must give it the terminal via tcsetpgrp(). After a foreground pipeline finishes or stops, the shell must reclaim the terminal via tcsetpgrp(). Background jobs must never receive the terminal, so Ctrl-C and Ctrl-Z only affect the foreground pipeline. On Ctrl-Z, the shell must wait using WUNTRACED, mark the pipeline's process group as Stopped, and print [job_number] + Stopped command, then return to the prompt. On Ctrl-D at the prompt (EOF on stdin), the shell must exit. If any job is Stopped when Ctrl-D is pressed, the shell must print cshell: there are stopped jobs and not exit. If Ctrl-D is pressed again immediately afterward (no other input in between), the shell must exit. When the shell exits (via Ctrl-D or otherwise) while background or stopped jobs still exist, the shell must send SIGHUP to every tracked job's process group before exiting. The shell must not wait for these processes to terminate. Example <goatam@iit:~> sleep 100^C<goatam@iit:~> sleep 100^Z[1] + Stopped sleep 100<goatam@iit:~> sleep 100 &[2] 4021<goatam@iit:~> ^C<goatam@iit:~> E3: resume [12] Syntax: resume %job_number (fg [-timeout <seconds>] | bg) Purpose: resume continues a stopped or backgrounded job, either bringing it to the foreground or letting it keep running in the background. Requirements job_number refers to a job as listed by activities. The shell must send SIGCONT to the job's process group. fg: the shell must give the job the terminal via tcsetpgrp(), mark it Running, wait for it to finish or stop again, then reclaim the`

terminal. `bg`: the shell must mark the job Running and return to the prompt immediately, without waiting or touching the terminal. If `--timeout <seconds>` is given with `fg`, the shell must start a timer using `alarm()` or `setitimer()` before waiting on the job. If the job is still running when the timeout elapses, the shell must send it `SIGTERM`, print `resume: job timed out`, and reclaim the terminal. If the job finishes or stops on its own before the timeout, the shell must cancel the pending timer. If `job_number` does not correspond to a known job, print `resume: no such job`. If the syntax does not match (missing job number, missing/invalid `fg/bg`, `--timeout` given without a valid number), print `resume: invalid syntax`. `bg`: after marking the job Running, print `[job_number] + Running command`. `fg`: before waiting on the job, print the job's command line (e.g. `sleep 100`), same as a normal foreground launch would. Example `<goatam@iit:~> sleep 100^Z[1] + Stopped sleep 100<goatam@iit:~> resume %1 bg[1] + Running sleep 100<goatam@iit:~> resume %1 fgsleep 100<goatam@iit:~> sleep 100^Z[1] + Stopped sleep 100<goatam@iit:~> resume %1 fg --timeout 3sleep 100resume: job timed out<goatam@iit:~> E4: ping [8] Syntax: ping <target> <signal_number> Purpose: The ping command sends a signal to a process or an entire process group. Requirements <target> is a plain number, interpreted as a pid, unless prefixed with %, in which case it is interpreted as a job number from activities and the signal is sent to every process in that job's group. The signal actually sent is signal_number % 64. signal_number is validated before <target> is looked up. If it fails validation (requirement 6), the syntax error is printed regardless of whether <target> would have resolved. If <target> does not correspond to any known process or job, print ping: no such process found. On success, print Sent signal signal_number to <target>. The message always echoes the original typed signal_number, not the value after % 64. If signal_number is not a valid non-negative integer, print ping: invalid syntax. Negative numbers are invalid syntax, not a valid signal value to be reduced by the modulo. ping may only target pids or jobs the shell itself has spawned and is still tracking. A pid that exists on the system but wasn't launched by this shell must be treated as unknown and print ping: no such process found. Example <goatam@iit:~> cat | sort &[2] 4030<goatam@iit:~> ping 4030 15Sent signal 15 to 4030<goatam@iit:~> ping %2 9Sent signal 9 to %2<goatam@iit:~> ping 4030 79Sent signal 79 to 4030<goatam@iit:~> ping 99999 9ping: no such process found<goatam@iit:~> ping %5 9ping: no such process found<goatam@iit:~> ping 4030 abcping: invalid syntax<goatam@iit:~> ping 4030 -3ping: invalid syntax Part F: Fun Stuff [20] F1: spy [10] Syntax: spy [pid] Purpose: spy lists information about open files associated with a given process to the standard output. An open file may be a regular file, a directory, a block special file, a character special file, an executing text reference, a library or a stream. Requirements: If no process ID is provided by the user, display the open files of the running shell. If a process ID is provided, display the open files of the specified process. For each entry, display: PID , FD , TYPE , PATH. FD should display the following: cwd: current working directory; mem: memory-mapped file (note that each unique memory mapped file should be printed only once); txt: executable file; numeric file descriptors such as 0, 1, 2 when they are open. Refer the example below. The TYPE field must correctly identify the file type of the object associated with each entry (e.g. REG, DIR, CHR, etc). Note that network specific file type objects are out of the scope of this mini-project. PATH must contain the path associated with the corresponding open object represented by FD. If the given process ID does not correspond to an existing process, print: spy: no such process. If more than one process IDs are passed as arguments, print: spy: invalid syntax. Hint: Process information can be accessed at /proc filesystem. Explore the directories in it to obtain the corresponding information. Think about how you can use readlink() for PATH. Isof source code may be a useful resource for understanding how such information can be obtained. Example: <goatam@iit:~> spy 21PID FD TYPE PATH21 cwd DIR /home/goatam21 txt REG /usr/bin/sleep21 mem REG /usr/lib/locale/locale-archive21 mem REG /usr/lib/libc.so.621 mem REG /usr/lib/gconv/gconv-modules.cache21 mem REG /etc/ld.so.cache21 mem REG /usr/lib/ld-linux-x86-64.so.221 0 CHR /dev/pts/0 F2: snoop [10] Syntax: snoop command [args...] or snoop -p pid Purpose: snoop traces a process's syscalls and reports a summary once it exits. Requirements If a command is given, the shell must fork, call PTRACE_TRACEME in the child, then execve the command. If -p pid is given, the shell must attach to the running process using PTRACE_ATTACH. Use PTRACE_SYSCALL to stop the tracee at each syscall entry and exit. Record the syscall number and a timestamp at entry, and a timestamp at exit, to compute time spent per call. Once the traced process exits, print a summary table: syscall name, number of calls, total time spent in it. Sort the summary by call count descending. Break ties by order of first occurrence. Syscall numbers not in your lookup table must print as syscall_N, where N is the syscall number. If the given pid does not exist, print snoop: no such process. If the command to launch does not exist, print snoop: command not found Example <goatam@iit:~> snoop sleep 1syscall calls timenanosleep 1 1.000swrite 1 0.000sexit_group 1 0.000s<goatam@iit:~> snoop -p 99999snoop: no such process Conclusion yay! you have a cshell now!`

Implementing a Multi-Level Feedback Queue (MLFQ) Scheduler in xv6 The default scheduling policy in xv6 is round-robin (RR). In this mini-project, you will implement an additional scheduling policy, the Multi-Level Feedback Queue (MLFQ), and integrate it into the xv6 kernel. The kernel must use only one scheduling policy, selected at compile time. If no policy is specified, the kernel must default to the original round-robin scheduler. We expect you to have finished the DIY section of Homework 2 (FIFO scheduler for xv6) before attempting this part of the mini project.

1. Build System Changes Modify the Makefile to support a `SCHEDULER` macro that selects the scheduling algorithm to compile in. Supported flag: `FlagSchedulerMLFQ` Multi-Level Feedback Queue If `SCHEDULER` is not passed, the build must default to plain round-robin. Example build commands: `# Default (Round Robin)make clean; make qemu# MLFQmake clean; make qemu SCHEDULER=MLFQ` An unset `SCHEDULER` must compile the original RR code path unchanged.

2. Scheduling Specification [100] 2.1 MLFQ [70] 2.1.1 Queues Implement four priority queues, numbered 0 (highest) to 3 (lowest). 2.1.2 Time Slices PriorityTime Slice (timer ticks)011428316 A "tick" refers to a clock/timer interrupt (see `kernel/trap.c`). 2.1.3 Scheduling Rules New process placement: On creation, a process is pushed to the end of queue 0. Strict priority selection: The scheduler always runs a process from the highest-priority non-empty queue. Example: a process running in queue 2 (with queues 0 and 1 empty) must be preempted if a new process arrives in queue 0. Preemption only needs to occur at tick boundaries — i.e., when the kernel next regains control of the CPU (end of the current tick). Process completion: When a process exits, it is removed from the queuing system. Time-slice exhaustion: If a process uses its entire time slice for its current queue, it is preempted and moved to the end of the next lower queue. If it is already in queue 3 (the lowest), it is re-inserted at the end of queue 3. Voluntary yield (e.g., I/O): If a process voluntarily gives up the CPU before its slice expires, it leaves the queuing network. When it becomes runnable again, it is inserted at the tail of the same queue it was in when it yielded (its priority is unchanged). Lowest queue behavior: Queue 3 is scheduled round-robin. Priority boosting (anti-starvation): Every 48 ticks, all processes in the system are moved to queue 0, regardless of their current queue.

2.1.4. Debugging Support: `procdump` `kernel/proc.c` contains `procdump()`, which is invoked when the user presses `Ctrl+P` on the console. Extend `procdump()` to print, for each process: its PID, name, state, current priority/queue number, and any other bookkeeping data relevant to verifying scheduler correctness (e.g. ticks consumed in current slice, ticks since last boost). Use this output while developing to confirm processes are moving between queues, preempting, and being boosted as specified. You may add any helper printing/logging code needed for testing; this is not restricted to the exact format already in `procdump`.

2.2. Cross-Scheduler Comparison [20] Compare FIFO, Round Robin (RR), and MLFQ using the

following metrics, computed over the same fixed workload/process set: *Metric* *Meaning* *Goal* *Turnaround Time* Time from process arrival to completion *Minimize* $\downarrow$ *Waiting Time* Total time spent waiting in the ready queue *Minimize* $\downarrow$ *Response Time* Time from arrival until the process first gets the CPU *Minimize* $\downarrow$ **2.3 Report Requirements [10]** Submit a short report (PDF or Markdown) containing: **2.3.1 Implementation Summary** For each specification item above (Makefile/SCHEDULER macro, struct proc changes, allocproc() changes, queue selection/preemption logic, time-slice handling, voluntary yield handling, priority boosting, procdump changes), write a few sentences explaining what you changed and why. **2.3.2 MLFQ Analysis** schedulertest is not part of default xv6: you will need to write it yourself as a custom user-space test program that spawns multiple child processes to exercise your scheduler. Use it to create several test processes that consume the CPU for varying amounts of time before voluntarily giving it up (e.g. simulate different CPU-burst lengths). Produce a timeline / scatter plot: X-axis: time elapsed (in ticks) since scheduler start. Y-axis: queue ID (0–3) the process is currently in. Color-coded by process (one color per PID/process). The plot must clearly show processes moving between queues over time and the periodic priority boost (all active processes jumping back to queue 0 every 48 ticks). Include the graph and 3–5 sentences interpreting it (e.g., which processes behave as CPU-bound vs. I/O-bound, and how the boost affects them). **2.3.3 Comparison Results** A table (or bar chart) reporting average Turnaround Time, Waiting Time, and Response Time for each of the three schedulers, on an identical set of test processes. A short discussion (5–8 sentences) on the trade-offs observed — e.g., why MLFQ may have lower response time than FIFO, why RR's waiting time depends on quantum size, etc. [PASTE YOUR PROJECT INSTRUCTIONS HERE]

Response: Welcome to the world of systems programming! Building a shell from scratch is a rite of passage for computer science students. It forces you to understand exactly how the OS manages processes, memory, and file descriptors.

Since you requested guidance specifically tailored to **Parts A, B, and C** (and skipping the rest), we will focus on designing a rock-solid foundation for parsing, built-ins, execution, and IPC (pipes/redirection).

Here is your architectural roadmap.

1. System Calls & Standard Library Functions (Parts A, B, C)

Here are the POSIX APIs you will need to research and use, categorized by purpose:

- **Process Control:** fork, wait, waitpid, exit
- **Execution (Part C only):** execv, execvp, execve
- **File I/O & Manipulation:** open, close, read, write, lseek (crucial for peek -r), stat, lstat, access
- **Directory & File System:** getcwd, chdir, opendir, readdir, closedir
- **Pipes & Redirection:** pipe, dup2
- **System Info:** gethostname, getuid, getpwuid (for user/host info in the prompt)
- **Environment:** getenv (to read PATH for locate and command execution)

2. High-Level Architecture

Do not write a monolithic shell.c. Instead, separate your concerns. A standard, extensible shell architecture looks like this:

- **main.c:** The entry point. Contains the main REPL (Read-Eval-Print Loop). It calls the prompt generator, gets input, and passes it to the parser.
- **prompt.c / prompt.h:** Handles the logic for username@hostname:path, properly substituting the home directory with ~.
- **lexer.c / lexer.h:** Converts raw string input into a stream of tokens (e.g., WORD, OP_PIPE, OP_GT). Handles quotes and escapes.
- **parser.c / parser.h:** Consumes tokens and builds a data structure representing the command(s) to execute. Validates syntax.
- **executor.c / executor.h:** Walks the parsed data structure, sets up pipes/redirections, forks children, and routes to either built-ins or execvp.
- **builtins.c / builtins.h:** The implementations for hop, reveal, peek, and locate. You might even split these into individual files (e.g., cmd_hop.c) if they get large.
- **utils.c / utils.h:** Helper functions for string manipulation, freeing memory, and error printing.

3. Data Structures

Your parser shouldn't just pass strings to your executor. It needs to build a structured representation of the user's intent.

Token Structure (Lexer output):

```
typedef enum {
    TOK_WORD, TOK_PIPE, TOK_SEMI, TOK_AMP,
    TOK_LT, TOK_GT, TOK_GTGT, TOK_EOF
} TokenType;

typedef struct {
    TokenType type;
    char *value; // Populated only if type is TOK_WORD
} Token;

// Represents a single process in a pipeline: e.g., "cat < a.txt > b.txt"
typedef struct {
    char **args;           // Array of strings (NULL terminated for execvp)
    int argc;              // Number of arguments
    char **input_files;    // List of files to read from (for <)
    char **output_files;   // List of files to write to (for > and >>)
    int *append_modes;     // Boolean array: 1 if >>, 0 if >

    // Hint: Storing multiple files handles cases like: command < f1 < f2
} Command;

// Represents an entire pipeline: e.g., "cat a.txt | grep foo"
typedef struct {
    Command *commands;     // Array of Command structs
    int num_commands;      // How many commands are in the pipe sequence
    int is_background;     // Flag for '&' (useful for Part D later)
} Pipeline;
```

Frecency Structure (for hop):

```
typedef struct {
    char *path;
    int score; // Calculated based on frequency and recency
} FrecencyEntry;
```

4. Component Breakdown & Logical Flow

Part A: Shell Input & Parsing

- Read:** Use `getline()` in a loop to read arbitrary-length user input. If it returns EOF (Ctrl+D), exit cleanly.
- Lexer (The State Machine):**
 - Iterate character by character.
 - Keep track of states: `NORMAL`, `IN_SINGLE_QUOTES`, `IN_DOUBLE_QUOTES`.
 - Apply maximal munch: If you see `>`, check the next character. If it's `>`, yield `TOK_GTGT`. Otherwise, yield `TOK_GT`.
- Parser:**
 - Iterate through the token array.
 - When you see a `WORD`, add it to the current Command's `args`.
 - When you see a redirection token (`<`, `>`), expect the next token to strictly be a `WORD` (the filename). If it isn't, throw a syntax error.
 - When you see `|`, finalize the current Command and start the next one in the Pipeline.

Part B: Shell Intrinsic

Remember: You cannot use `exec*` here!

- hop:** Use `chdir()`. Keep track of the previous directory in a static variable (or environment variable) for `hop`. For frecency, write a hidden file (e.g., `~/.cshell_frecency`) where you serialize your paths and scores. Load it on startup, update on `hop`, and save.
- reveal:** Use `opendir()` and `readdir()`. Store the results in an array of strings, then use `qsort()` to sort them lexicographically. For `-t` (tree/recursive), write a recursive function that explores directories, keeping track of indentation depth.

- **peek:** Standard reading is trivial. For `-r` (reverse) on regular files: Use `lstat()` to get file size. Set a buffer (e.g., 4KB). Use `lseek()` to jump to size `- 4096`, read the chunk, search for `\n` backwards, and print lines. Repeat until you reach the start of the file.
- **locate:** Get `$PATH`. Use `strtok()` to split it by `:`. For each directory, append `/command_name` and use `access()` or `stat()` with the `X_OK` flag to see if it's an executable file.

Part C: Execution, Redirection, and Pipes

This is the most critical part of an OS project.

1. Redirections (`<`, `>`, `>>`):

- Before calling `execvp` in the child process, open the files.
- For input: `fd = open(file, O_RDONLY)`. Then `dup2(fd, STDIN_FILENO)`.
- For output: `fd = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644)`. Then `dup2(fd, STDOUT_FILENO)`.
- Close `fd` after duplicating.

2. Pipes (`|`):

- If a pipeline has N commands, you need $N - 1$ pipes.
- Loop through commands $i = 0$ to $N - 1$.
- Create a pipe using `pipe(fd_array)`.
- `fork()`.
- **In the Child:**
 - If not the first command, wire `STDIN` to the read end of the *previous* pipe.
 - If not the last command, wire `STDOUT` to the write end of the *current* pipe.
 - Apply file redirections (these overwrite pipe endpoints, which is correct behavior!).
 - **CRITICAL:** Close all unused pipe file descriptors in the child, then `execvp`.
- **In the Parent:**
 - Close the write end of the current pipe (the parent doesn't write to it).
 - Save the read end of the current pipe to become the "previous pipe" for the next iteration.
- Wait for all children in the pipeline using `waitpid()`.

5. Edge Cases & Pitfalls to Avoid

1. **The "Dangling Pipe" Hang:** If your parent process forgets to close the write end of a pipe, the child reading from that pipe will *never* receive an EOF. A command like `ls | sort` will just hang forever because `sort` is waiting for the parent to write something.
2. **Memory Leaks:** Shell loops run indefinitely. If you `malloc` strings in your lexer and AST nodes in your parser, you **MUST** write a recursive `free_pipeline()` function and call it at the very end of your REPL loop.
3. **String Truncation & Paths:** Spaces in paths are tricky (e.g., `cat my\ file.txt`). Your lexer must handle the `\` escape character so that `my file.txt` is treated as a *single* token `WORD`, not two.
4. **peek -r on non-seekable files:** You cannot `lseek()` on a pipe (e.g., `cat | peek -r`). You must check if the file descriptor is seekable. If it isn't, you must buffer the entire input in memory before reversing it.
5. **hop Edge Case:** If a user runs `hop dir1`, then `hop dir2`, then `hop -`, it should go to `dir1`. If they run `hop - again`, it should go back to `dir2`. The "previous CWD" toggles between the two.

Start with Part A. Do not move to Part B until your prompt prints cleanly and your parser can correctly reject invalid syntax with `cshell: invalid syntax`. Good luck!

User prompt: Give a much more detailed breakdown of A1

Response: Here is a deep dive into architecting Part A1: The Shell Prompt.

While printing a prompt seems trivial, doing it robustly in C requires careful memory management, string manipulation, and handling of global state.

1. Required POSIX System Calls & Headers

To extract the necessary information for `<username@hostname:currentpath>`, you will need to research these specific APIs:

- **getuid()** (Header: `<unistd.h>`): Gets the real user ID of the calling process.
- **getpwuid()** (Header: `<pwd.h>`): Takes a user ID and returns a pointer to a `struct passwd`, which contains the `pw_name` (the username as a string). This is much safer and more POSIX-compliant than using `getenv("USER")`.
- **gethostname()** (Header: `<unistd.h>`): Retrieves the standard host name of the current machine.
- **getcwd()** (Header: `<unistd.h>`): Copies the absolute pathname of the current working directory into a buffer you provide.

2. State Management: The Shell's "Home"

The specification states: *"The directory the shell is started in becomes its home directory."*

This means you cannot rely on the system's `$HOME` environment variable. You must capture the shell's starting directory the moment your program begins and persist it for the lifetime of the shell.

Architectural Advice: Create a global or session-level state struct to hold this. Do this as the very first step in your `main()` function before your REPL loop starts.

```
// High-level conceptual structure (e.g., in shell.h)
typedef struct {
    char base_dir[PATH_MAX]; // The directory where ./shell.out was executed
    char hostname[HOST_NAME_MAX];
    char username[LOGIN_NAME_MAX];
} ShellContext;
```

Note: `PATH_MAX`, `HOST_NAME_MAX`, and `LOGIN_NAME_MAX` are macros found in `<limits.h>`.

Populate the `hostname` and `username` during this initialization step as well. They will never change during the shell's execution, so fetching them once saves unnecessary syscall overhead in your loop.

3. Step-by-Step Logical Flow (The Prompt Generator)

Every time your shell completes a foreground process (or starts up), you will call your prompt generation function. Here is the logical breakdown:

Step 1: Fetch the Current State Use `getcwd()` to grab the actual, current absolute path where the user is sitting right now.

Step 2: The Prefix Matching Logic (The tricky part) You need to determine if the `current_path` is a child of the `base_dir` (the shell's home). If it is, you substitute the `base_dir` portion with `~`.

Pseudocode for the logic:

```
IF length of current_path >= length of base_dir:
    IF the first (length of base_dir) characters of current_path MATCH base_dir:

        // CRITICAL CHECK: Ensure we are at a directory boundary!
        char next_char = current_path[length of base_dir]

        IF next_char is '\0' OR next_char is '/':
            // We have a valid sub-directory match.
            // Print the prompt using '~' instead of the base_dir
            Print: "<[user]@[host]:~[remainder of current_path]>"
            RETURN

// Fallback: It is NOT a subdirectory of the shell's home.
Print: "<[user]@[host]:[current_path]>"
```

Step 3: Pointer Arithmetic for String Slicing To print the "remainder of `current_path`" without copying strings around in memory, use C

pointer arithmetic. If `base_dir` is `/home/student/osn` and `current_path` is `/home/student/osn/src`, the remainder is `/src`.

You can simply pass `current_path + strlen(base_dir)` to your print function to skip the prefix!

4. Edge Cases & Pitfalls to Watch Out For

- **The "Substring Illusion" Bug:** This is the most common mistake students make here.
 - Let's say `base_dir` is `/home/user/project`.
 - The user hops to an entirely different directory: `/home/user/project_two`.
 - If you just use `strncmp` to check the prefix, `/home/user/project` matches the first part of `/home/user/project_two`.
 - Without the **"Directory Boundary Check"** (checking if the next character is `\0` or `/`), your prompt will incorrectly print `<username@host:~_two>`.
- **Buffer Overflows:** Never use unbounded string functions. Always pass the `PATH_MAX` or `HOST_NAME_MAX` limits into `getcwd` and `gethostname` to ensure they don't overwrite memory if a path is bizarrely long.
- **Terminal Colors (Optional but helpful):** Though not strictly in the spec, you can use ANSI escape codes in your `printf` statement to make the prompt stand out (e.g., making the path blue and the user green). This makes debugging your shell output much easier later on when commands start spilling lots of text onto the screen.

User prompt: A2

Response: Part A2 ("User Input") might look like the easiest section of the entire project—after all, it's just reading a string, right? However, in C, reading unbounded user input is the number one source of buffer overflows, memory leaks, and segmentation faults.

Here is the architectural breakdown of how to handle user input robustly, setting up the central heartbeat of your shell: the **REPL (Read-Eval-Print Loop)**.

1. Required POSIX Functions

To read arbitrary-length input safely, you should rely on this specific POSIX extension (which is standard in modern C environments):

- **`getline()`** (Header: `<stdio.h>`): This function reads an entire line from a stream (like `stdin`), automatically allocating and re-allocating memory as needed if the user types a very long line.
- **`free()`** (Header: `<stdlib.h>`): Because `getline` manages memory for you dynamically, you must take responsibility for freeing it.

Strictly Avoid: Never use `gets()` (it is notoriously unsafe and removed from modern C). Avoid `scanf("%s")` as it stops at spaces. You *can* use `fgets()`, but it requires a fixed-size buffer; if the user types a command longer than your buffer, you'll have to stitch it together manually. `getline()` is the superior architectural choice here.

2. The REPL Architecture (The Main Loop)

Your `main.c` should contain an infinite loop. This loop is the conductor for your entire project. For Part A2, the architecture of this loop looks like this:

Initialize `ShellContext` (Get home directory, hostname, username)

LOOP FOREVER:

1. Print Prompt (Call your A1 function)
2. Read Input (Call your A2 function)
 - If EOF (Ctrl+D), EXIT loop cleanly.
 - If error, handle it.
3. Sanitize Input
 - Strip trailing newline.
 - If input is empty, CONTINUE loop (print prompt again).
4. (Later Parts) Pass to Lexer -> Parser -> Executor
5. Cleanup memory allocated during this iteration

3. Step-by-Step Logical Flow for `getline`

The `getline` function requires three parameters: a pointer to a string (which will hold the text), a pointer to a `size_t` (which tracks the size of the allocated buffer), and the input stream (`stdin`).

The Setup Flow:

1. Initialize a `char *buffer = NULL`;
2. Initialize a `size_t bufsize = 0`;
3. Call `getline(&buffer, &bufsize, stdin)`;
 - *Why pass by reference?* If `buffer` is `NULL`, `getline` will `malloc` the exact amount of space needed for you and update `buffer` to point to it. If the user keeps typing, it will `realloc` the buffer and update `bufsize`.
4. Check the return value of `getline`. It returns the number of characters read, or `-1` on failure or EOF.

The Cleanup Flow: Once you are completely done processing the user's command for that iteration of the REPL, you must call `free(buffer)` before the loop starts over. Alternatively, you can reuse the same buffer across loop iterations and only `free` it when the shell exits, which is slightly more performant.

4. Edge Cases & Pitfalls to Watch Out For

- **The Trailing Newline (`\n`):** When a user types `ls` and hits Enter, `getline` captures exactly what they typed: `ls\n`. If you pass `ls\n` to your parser (or later, to `execvp`), the OS will look for a command literally named `ls\n` and fail.
 - *Fix:* You must inspect the string and replace the trailing `\n` with a null terminator (`\0`).
- **The Infinite EOF Loop:** If a user presses `Ctrl+D`, `getline` returns `-1`. If you don't explicitly check for this return value and break out of your loop, your shell will spam the prompt infinitely because `stdin` has been closed.
- **Empty Input:** If the user just hits Enter, the buffer will contain just `\n` (which you will convert to `\0`). Your code needs to recognize this as an empty command and continue to the next iteration of the loop rather than passing an empty string to the lexer/parser.
- **Interrupted System Calls (Part E foreshadowing):** Later in the project, when you implement signal handlers (`Ctrl+C` / `Ctrl+Z`), `getline` might return `-1` not because of EOF, but because a signal interrupted it (setting `errno` to `EINTR`). You will need to clear the error state of `stdin` using `clearerr(stdin)` and loop back to the prompt instead of exiting. Keeping your reading logic modular now will make this addition easy later.

User prompt: `A3`

Response: Part A3 ("Input Parsing") is the most intellectually demanding piece of the entire project. If you build this correctly, Parts C, D, and E will snap into place easily. If you hack it together with `strtok()`, your shell will completely collapse when you try to handle quotes, escapes, and complex pipelines.

Because the project specification explicitly defines a regular grammar and character classes, you must build this in two distinct phases: **The Lexer (Tokenizer)** and **The Parser (Validator & AST Builder)**.

Here is your architectural blueprint for building them.

1. Required C Library Functions

You will not need OS syscalls here, but you will heavily rely on the C standard library for memory and string manipulation:

- **Memory Management:** `malloc`, `realloc` (crucial for dynamically growing token arrays and strings), `calloc`, `free`.
- **String Tools:** `strchr` (great for checking if a character is in a set of special characters), `strlen`, `strncmp`.

2. Phase 1: The Lexer (State Machine)

The Lexer's job is to read the raw input string character by character and convert it into a flat array (or linked list) of Token structs. **It does not care if the command makes sense** (e.g., `| < | >` is perfectly fine to the lexer). It only cares about grouping characters into valid words or operators.

Data Structures:

```
// Define the token types based on the spec
typedef enum {
    TOK_WORD, TOK_PIPE, TOK_AMP, TOK_SEMI,
    TOK_LT, TOK_GT, TOK_GTGT, TOK_EOF
} TokenType;

typedef struct {
    TokenType type;
    char *value; // dynamically allocated string (only for TOK_WORD)
} Token;
```

Architectural Logic (The Finite State Machine): Do NOT use strtok. You need a while loop that iterates through the string one character at a time, keeping track of its "State".

Pseudocode for Lexer Loop:

```
State = NORMAL
Initialize an empty string buffer (to build the current WORD)
Initialize a Token list

FOR EACH character 'c' in input:
    SWITCH (State):
        CASE NORMAL:
            IF c is Space/Tab/Newline:
                If buffer is not empty, save buffer as TOK_WORD, clear buffer.
            ELSE IF c is Special ('|', '&', ';', '<', '>'):
                If buffer is not empty, save buffer as TOK_WORD, clear buffer.
                Check next character for Maximal Munch!
                (If c == '>' AND next_char == '>', yield TOK_GTGT, skip next char)
                (Else yield the single character token)
            ELSE IF c is Single Quote ('\'):
                State = IN_SINGLE_QUOTE
            ELSE IF c is Double Quote (''):
                State = IN_DOUBLE_QUOTE
            ELSE IF c is Escape ('\'):
                Check next character. If it's the end of string -> THROW LEXICAL ERROR
                Add the NEXT character literally to buffer, skip it in loop.
            ELSE:
                Add c to buffer.

        CASE IN_SINGLE_QUOTE:
            IF c is Single Quote ('\'):
                State = NORMAL
            ELSE:
                Add c literally to buffer (NO escape processing!)

        CASE IN_DOUBLE_QUOTE:
            IF c is Double Quote (''):
                State = NORMAL
            ELSE IF c is Escape ('\'):
                Check next character.
                If next is '' or '\', add next character to buffer, skip it.
                Else add '\' AND next character to buffer.
            ELSE:
                Add c to buffer.

// End of loop checks
IF State != NORMAL:
    THROW LEXICAL ERROR ("cshell: invalid syntax") // Unclosed quote!
IF buffer is not empty:
    Save buffer as TOK_WORD
Add TOK_EOF to token list
```

3. Phase 2: The Parser (Grammar Validation)

The Parser takes your list of tokens and walks through them to ensure they obey the right-linear grammar provided in the spec. While validating, it builds the Pipeline and Command structures we discussed previously.

Architectural Logic (Token Walking): Since it's a right-linear grammar, you can validate this with a simple while loop over your token array.

Pseudocode for Parser Validation:

```
Expect_Command = TRUE // We expect a WORD to start a command
Current_Command = Create new Command struct

FOR EACH token in Token List:
    IF Expect_Command is TRUE:
        IF token is TOK_WORD:
            Add token to Current_Command args
            Expect_Command = FALSE
        ELSE IF token is TOK_EOF:
            BREAK // Empty line, perfectly valid!
        ELSE:
            THROW_PARSER_ERROR ("cshell: invalid syntax") // e.g., starting with '|'

    ELSE (Expect_Command is FALSE):
        IF token is TOK_WORD:
            Add token to Current_Command args (it's just another argument)

        ELSE IF token is TOK_LT, TOK_GT, TOK_GTGT:
            Look at the NEXT token.
            IF next token is NOT TOK_WORD:
                THROW_PARSER_ERROR ("cshell: invalid syntax") // e.g., '<|`
            Add next token as input/output file to Current_Command based on operator.
            Skip the next token in the loop.

        ELSE IF token is TOK_PIPE:
            Finalize Current_Command, add to Pipeline.
            Expect_Command = TRUE // Next token MUST start a new command

        ELSE IF token is TOK_SEMI OR TOK_AMP:
            Finalize Current_Command, add to Pipeline.
            Mark Pipeline as background if TOK_AMP.
            Expect_Command = TRUE // Next token starts a totally new sequence
```

4. Edge Cases & Pitfalls to Watch Out For

1. **The "Maximal Munch" Trap:** When parsing >>, you must consume both > characters immediately. If you parse it as two TOK_GT tokens, your parser will expect a filename after the first >, see the second >, and crash with a syntax error.
2. **Empty Quotes (" or '):** echo "" should result in an argument array of {"echo", "", NULL}. If your lexer completely drops empty quotes, your parser will think the user just typed echo. Your buffer logic must yield a TOK_WORD containing an empty string \0 if a quote is opened and immediately closed.
3. **Whitespace Inside Quotes:** cat "my file.txt" should yield exactly *two* tokens: cat and my file.txt. Do not accidentally split at the space inside the quotes.
4. **Memory Management (The silent killer):** You are building an array of tokens (where each word is a malloc'd string), and then an AST (where each Command holds arrays of malloc'd strings). You must write a robust free_tokens() and free_pipeline() function early. Run your code through valgrind right now. Do not wait until Part C to check for memory leaks!
5. **Grammar Edge Case (& & or | ;):** If the user types echo hi & &, the first & sets Expect_Command = TRUE. The loop looks at the second &, sees that a command was expected, and correctly throws "cshell: invalid syntax". Ensure your state booleans strictly enforce what can follow what.

User prompt: B1

Response: Part B1 (hop) introduces a crucial concept in OS programming: maintaining environmental state and persisting data across process executions.

Unlike most commands which you will eventually fork and exec, hop **must** be implemented as a shell built-in. If you fork a child process to run chdir(), only the child's working directory changes; the parent shell remains exactly where it was.

Here is the architectural breakdown for building hop and its frequency engine.

1. Required POSIX & C Library Functions

- **Directory Control:** `chdir()` (to actually move), `getcwd()` (to find out where you landed).
- **File System Checks:** `access()` or `stat()` (to check if a directory from your frecency DB still exists on disk).
- **String Manipulation:** `strstr()` (crucial for finding if the argument is a substring of a saved path).
- **File I/O (Persistence):** `fopen()`, `fprintf()`, `getline()` (to read/write your frecency database file).
- **Time:** `time()` (Header: `<time.h>`) to track the "recency" portion of frecency.

2. State & Data Structure Design

You need to expand the `ShellContext` we discussed in Part A1 to track the "previous" directory. You also need a data structure for the Frecency Database.

```
// Add to your global/session state (e.g. ShellContext)
char previous_cwd[PATH_MAX]; // Initialized to empty string '\0'

// Structure for a single frecency record
typedef struct {
    char path[PATH_MAX];
    int frequency;           // How many times visited
    long last_visit;        // UNIX timestamp of last visit
    double score;           // Calculated dynamically
} FrecencyRecord;

// Structure for the DB in memory
typedef struct {
    FrecencyRecord *records; // Dynamically allocated array
    int count;
    int capacity;
} FrecencyDB;
```

3. Component Breakdown & Logical Flow

The `hop` command handles an arbitrary number of arguments. Your executor should loop through every argument passed to `hop` and execute the following flow sequentially.

Step 1: Determine the Target Path (Resolution) For each argument `arg` (or `~` if no arguments):

1. **If `arg == ~` or `NULL`:** Target is `ShellContext.base_dir`.
2. **If `arg == .`:** Target is the current directory (do nothing, but maybe update frecency).
3. **If `arg == ..`:** Target is `..` (handled natively by `chdir`).
4. **If `arg == -`:** Target is `previous_cwd`. If `previous_cwd` is empty (no hops yet), do nothing and continue to the next argument.
5. **Otherwise (name):**
 - First, try to use `arg` as a direct path.
 - If `chdir(arg)` succeeds, you are done with resolution.
 - If `chdir(arg)` fails, **DO NOT THROW AN ERROR YET**. This triggers the Frecency Fallback (Step 2).

Step 2: The Frecency Fallback (When direct `chdir` fails) If the direct jump failed, `arg` is just a substring (like `osn`).

1. Calculate the score for every entry in your loaded `FrecencyDB`. A simple deterministic formula: $\text{Score} = \text{Frequency} * (1.0 / (\text{Current_Time} - \text{last_visit} + 1))$. (You can tweak this math).
2. Iterate through the DB. Find all records where `strstr(record.path, arg)` is not `NULL`.
3. Sort these matching records by score descending.
4. Iterate through the sorted matches:
 - Use `access(match.path, F_OK)` to see if it still exists on disk.
 - If it exists, `target = match.path`. Break the loop.

- If it doesn't exist, skip to the next best match.

5. If you exhaust the list with no valid directory, print "hop: no such directory" and halt execution for this argument.

Step 3: Execution and State Update Once you have a valid target path:

1. Call `getcwd` and save it to a temporary variable (`current_before_hop`).
2. Call `chdir(target)`.
3. If `chdir` succeeds:
 - Copy `current_before_hop` into `previous_cwd`.
 - Call `getcwd()` again to get your *new* absolute path.
 - **Update Frecency DB:** Search the DB for this new absolute path. If it exists, `frecency++` and update `last_visit`. If it doesn't, add a new record.
 - **Persist:** Write the entire in-memory DB back to your hidden disk file (e.g., `<shell_base_dir>/cshell_frecency`).

4. Edge Cases & Pitfalls to Watch Out For

1. **Multiple Arguments Behavior:** The spec allows `hop ~ . . - osn`. Your code must execute these *one after the other*. Do not just parse the last one. Every step in that chain updates the `previous_cwd` and the frecency database. If one fails (e.g., `osn` doesn't resolve), print the error but only for that specific jump.
2. **The Frecency Storage Location:** Where do you save the `.cshell_frecency` file? Do not save it in the directory where `hop` lands, or you'll leave database files scattered everywhere. Save it strictly in the `ShellContext.base_dir` (the directory where `./shell.out` was originally launched).
3. **File Format Parsing:** When reading your `.cshell_frecency` file on startup, be careful with string parsing. If your file format is `/path/to/dir,5,1698765432`, use `fscanf` or `strtok` carefully. Corrupted DB files shouldn't crash your shell; if parsing fails, just start with an empty DB.
4. **hop - Ping-Pong:** If the user types `hop -` over and over, they should bounce back and forth between two directories.
 - CWD: `/A`, Prev: `/B`
 - Run `hop -` -> Target is `/B`.
 - Before `chdir`, save CWD (`/A`).
 - After `chdir`, Prev becomes `/A`. CWD is `/B`.
 - Next `hop -` will correctly target `/A`.
5. **Memory Leaks in DB Loading:** If you dynamically resize your `FrecencyRecord` array, ensure you `free()` it entirely when the shell exits (on `Ctrl+D` / EOF).

User prompt: `B2`

Response: Part `B2(reveal)` requires you to interact directly with the OS's file system metadata. Since you cannot use `exec*` to call the system's `ls` or `tree` commands, you must build the directory traversal logic yourself.

Here is the architectural breakdown for implementing a robust `reveal` command.

1. Required POSIX System Calls & Headers

- **Directory Traversal:** `opendir`, `readdir`, `closedir` (Header: `<dirent.h>`).
- **File Metadata:** `stat` or `lstat` (Header: `<sys/stat.h>`). You will need this to check if a path actually exists before revealing it, and to check if a file is a directory when doing recursive reveals.
- **Path Resolution:** `access` (Header: `<unistd.h>`) to quickly verify if a direct path is valid.
- **Sorting:** `qsort` (Header: `<stdlib.h>`) for ASCII lexicographic sorting.

2. Phase 1: Parsing Flags vs. Paths

The grammar for reveal is: `reveal (- (a|t)*)* (~ | . | .. | - | name)?` This means you can have zero or more flag blocks, followed by *at most one* target path.

Architectural Logic: You need a parser loop just for this command. Set up two booleans: `bool flag_a = false;` and `bool flag_t = false;`. Set up a char `*target_path = NULL;`.

Iterate through `Command . args` (starting from index 1, since index 0 is "reveal"):

1. Is it a flag or a path?

- If the argument is exactly "- ", it is the TARGET PATH (previous working directory).
- If the argument starts with '-' and its length is > 1 (e.g., -a, -ta, -tttt), it is a FLAG BLOCK.
- Otherwise, it is the TARGET PATH.

2. Processing Flags: Loop through the characters of the flag block (skipping the first -).

- If 'a', `flag_a = true;`
- If 't', `flag_t = true;`
- If anything else, print "reveal: invalid syntax" and abort the command immediately.

3. Processing Path: If you identify a target path, assign it to target_path. If target_path is already not NULL (meaning the user provided multiple paths like `reveal dir1 dir2`), print "reveal: invalid syntax" and abort.

3. Phase 2: Target Path Resolution

If `target_path` is NULL after parsing, default it to "." (current directory). Otherwise, resolve the path using the exact same logic you built for `hop` (handling ~, ., .., -, and direct paths), **with one major difference:** Do not apply the Frecency fallback.

If the path is - and the previous directory is empty (no hops have occurred), print "reveal: no such directory" and abort.

Once resolved, use `stat()` to verify the directory exists. If `stat()` fails, print "reveal: no such directory".

4. Phase 3: The Directory Traversal Engine

Because of the -t (recursive) flag, your core logic must be implemented as a **recursive function**.

Pseudocode for `reveal_dir(char *base_path, bool show_hidden, bool recursive)`:

Step 1: Read and Filter

1. Call `opendir(base_path)`.
2. Loop `readdir()` until it returns NULL.
3. For each `struct dirent *entry`:
 - If !`show_hidden` AND the filename starts with . -> Skip it.
 - Otherwise, duplicate the filename into a dynamically allocated array of strings (`char **entries`).

Step 2: Sort

1. Use `qsort` on your `entries` array.
2. Your comparator function should just use standard `strcmp`. The spec demands sorting by ASCII value, which is exactly what `strcmp` does natively.

Step 3: Print and Recurse

1. **Print Phase:** Iterate through the sorted `entries` array.
 - Construct the relative path to print. For the top level, you print the filename. For subdirectories, you print `base_path/filename`.
 - (Note: Look closely at the spec's example output to format exactly as requested. Usually directories append a / at the end).

2. **Recursive Phase:** If recursive is true:

- Iterate through the entries array *again*.
- **CRITICAL:** If the entry name is "." or "..", skip it! (Otherwise, you will recurse infinitely).
- Construct the full absolute/relative path to this entry.
- Use `stat(full_path, &st)` to get its metadata.
- Use the `S_ISDIR(st.st_mode)` macro. If it is a directory, call `reveal_dir(full_path, show_hidden, recursive)`.

3. **Cleanup:** Free the duplicated strings in your entries array, free the array itself, and call `closedir()`.

5. Edge Cases & Pitfalls to Watch Out For

1. **The Infinite Recursion Trap:** The most common mistake in building `-t` is forgetting that *every* directory contains `.` (itself) and `..` (its parent). If you do not explicitly filter out `strcmp(name, ".") == 0` and `strcmp(name, "..") == 0` before doing the recursive step, your shell will crash with a stack overflow.
2. **d_type vs. stat():** The struct `dirent` returned by `readdir` has a `d_type` field that can tell you if an entry is a directory (`DT_DIR`). **However**, on some file systems (like XFS or certain network mounts), `d_type` returns `DT_UNKNOWN`. The architecturally safe OS method is to construct the full path and call `stat()`, then check `S_ISDIR(st_mode)`.
3. **String Concatenation for Paths:** When building paths for the recursive step, don't blindly `strcat`. If `base_path` is `include` and the file is `parser.h`, you need to format it as `include/parser.h`. Make sure you insert the `/`, but avoid double slashes (`//`) if `base_path` already ends in one. Use `snprintf()` for safe, formatted path construction.
4. **Memory Management:** You are reading an unknown number of files. You should either:
 - Do a two-pass approach: Read the directory once just to count the valid files, `malloc` the exact array size, then rewind the directory stream (`rewinddir()`) and read them in.
 - Or, use a dynamic array (a capacity variable that doubles with `realloc` when full). This is generally faster and preferred.

User prompt: B3

Response: Part B3 (peek) is your version of the classic UNIX `cat` command, but with a wicked twist. The `-r` (reverse) flag combined with the requirement to read backwards using fixed-size chunks via `lseek` is one of the most mechanically complex parts of this project.

Here is the architectural breakdown of how to build a rock-solid peek utility.

1. Required POSIX System Calls & Headers

- **File Operations:** `open`, `read`, `write`, `close` (Header: `<unistd.h>`, `<fcntl.h>`).
- **Cursor Control:** `lseek` (Header: `<unistd.h>`) - the star of the show for seekable backward reading.
- **File Metadata:** `fstat` (Header: `<sys/stat.h>`). Used to check if the file is a directory (`S_ISDIR`) and if it is a regular/seekable file (`S_ISREG`).

2. Phase 1: Parsing Flags vs. Files

Just like `reveal`, you need to parse the arguments to separate flags from target files.

Architectural Logic:

1. Initialize `bool flag_n = false;` and `bool flag_r = false;`.
2. Initialize a dynamic array or linked list for filenames.
3. Iterate through `Command.args` (starting at index 1):
 - If the argument is exactly `"-"`, treat it as a filename (meaning `stdin`).
 - If the argument starts with `'-'` and `length > 1`, parse characters:

- If 'n', flag_n = true;
- If 'r', flag_r = true;
- (Note: Unlike boolean toggles, -rr just means flag_r = true. They don't cancel each other out.)
- Otherwise, add it to the filenames list.

4. If the filenames list is empty after the loop, add "-" (stdin) as the default target.

3. Phase 2: File Iteration & Dispatcher

For each file in your filenames list, execute the following dispatch logic sequentially:

Step 1: Open and Validate

- If filename is "-", fd = STDIN_FILENO.
- Otherwise, fd = open(filename, O_RDONLY). If it fails, print "peek: no such file or directory" and move to the next file.
- Call fstat(fd, &st).
- If S_ISDIR(st.st_mode), print "peek: is a directory", close fd, and move to the next file.

Step 2: Determine Execution Path You have three highly distinct operational modes:

- **Mode A (Forward):** !flag_r. (Applies to both files and stdin).
- **Mode B (Backward Seekable):** flag_r AND S_ISREG(st.st_mode).
- **Mode C (Backward Non-Seekable):** flag_r AND !S_ISREG(st.st_mode) (e.g., pipes, stdin).

4. Phase 3: The Three Execution Engines

Mode A: Forward Reading (The Standard cat)

1. Read the file in chunks (e.g., char buffer[4096]) using read().
2. Maintain a line_counter (start at 1).
3. Iterate through the buffer. If flag_n is true, check if a line is "empty" (just a \n). If it has characters, print line_counter followed by the line, then increment line_counter.
4. *State Management:* You must maintain state across read() boundaries. If a line gets cut in half by your 4096-byte chunk, you shouldn't print the line number again on the next chunk!

Mode B: Backward Seekable (The lseek Engine)

This is where the spec tests your systems programming limits. You are strictly forbidden from loading the whole file into memory.

- **The -n Paradox:** If -n and -r are both true, the last line prints first. But what is its line number? To know that, you must know the *total* number of non-empty lines!
 - *Solution:* If flag_n is true, quickly scan the file *forward* using chunked read() just to count the total non-empty lines. Store this in total_lines.
- **The Chunked Reverse Read:**
 1. Use lseek(fd, 0, SEEK_END) to get the file size.
 2. Set current_offset = file_size.
 3. Loop while current_offset > 0:
 - Calculate chunk_size (min of 4096 and current_offset).
 - current_offset -= chunk_size.
 - lseek(fd, current_offset, SEEK_SET).

- `read(fd, buffer, chunk_size)`.
- Scan buffer *backwards* (from `chunk_size - 1` down to `0`).
- Every time you find a `\n`, print the characters from that `\n` to the end of the line.
- *Crucial:* Since chunks slice strings arbitrarily, you must buffer the "leftover" partial line at the start of the chunk, prepend it to the end of the *next* chunk you read, and process them together.

Mode C: Backward Non-Seekable (The Memory Buffer)

If you try to `lseek` on a pipe (like `echo hello | peek -r`), it will fail with `ESPIPE`. The spec explicitly allows buffering in memory for this.

1. Initialize a dynamically resizing buffer (using `malloc` and `realloc`).
2. `read()` in chunks, appending everything to your growing buffer until `read()` returns `0` (EOF).
3. Once the entire stream is in memory, traverse it backwards, locating `\n` boundaries, and print the lines (handling `flag_n` decrementing just like Mode B).
4. `free()` the buffer.

5. Edge Cases & Pitfalls to Watch Out For

1. **The "Missing EOF Newline" Bug:** Many text files do not end with a newline character. When reading backwards in chunks, if the very last byte of the file isn't `\n`, your logic might drop the final line entirely. Always check if there are trailing characters after the last `\n`.
2. **"Empty" Line Definition:** The spec says `-n` counts *non-empty* lines. A line is typically empty if it contains only `\n`. A line with spaces is technically not empty. Make sure your line-counting logic strictly aligns with this.
3. **Boundary Tearing in Mode B:** Imagine reading a 4096-byte chunk, and the chunk starts precisely in the middle of a word: `[... chunk n-1]hel | lo world\n [... chunk n]`. When reading chunk *n* backwards, you will see `lo world\n`. You must save `lo world` in a temporary string, read chunk *n-1*, find the `hel`, concatenate them to form `hello world`, and *then* print it.
4. **Preserving File Order in Multi-file Reverse:** If the user runs `peek -r fileA fileB`, print *all* of `fileA` in reverse, and *then* all of `fileB` in reverse. Do not concatenate them and reverse the master combination. Treat each file independently in your loop.

User prompt: `B4`

Response: Part `B4` (`locate`) acts very much like the classic `which` command (or `command -v`), but with a specific requirement to check the Current Working Directory (CWD) *before* checking the `PATH`, and to print *all* matching executable paths rather than stopping at the first one.

This component tests your ability to interact with the shell's environment variables and safely manipulate strings. Here is your architectural breakdown.

1. Required POSIX & C Library Functions

- **Environment Access:** `getenv` (Header: `<stdlib.h>`) to fetch the `PATH` variable.
- **File Verification:** `access` (Header: `<unistd.h>`). Specifically, use `access(filepath, X_OK)` to check if a file both exists and grants execute permissions to the current user.
- **Path Resolution:** `realpath` (Header: `<stdlib.h>`) or `getcwd` (Header: `<unistd.h>`) to ensure you are printing absolute paths as the spec demands.
- **String Manipulation:** `strdup` (crucial for environment safety), `strtok` or `strtok_r`, and `snprintf`.

2. Phase 1: Environment Setup & Optimization

The `PATH` environment variable is a single string with directories separated by colons (e.g., `/usr/local/bin:/usr/bin:/bin`).

Architectural Advice (The Pre-computation Step): If the user runs `locate cmd1 cmd2 cmd3`, you do not want to parse the `PATH` string three separate times. Worse, `strtok()` modifies the string it parses by inserting `\0` characters.

Before looping through the arguments, extract and parse the `PATH` once:

1. Call `getenv("PATH")`.
2. **CRITICAL:** Do NOT modify the pointer returned by `getenv`. It points directly into your process's environment block. Modifying it is undefined behavior. Use `strdup()` to create a copy on the heap.
3. Use `strtok()` on your copy with `":"` as the delimiter to split it.
4. Build an array of strings (`char **path_dirs`) containing these directories.

3. Component Breakdown & Logical Flow

For each filename argument passed to the `locate` command, you will execute a discrete search engine flow:

Step 1: Setup for the Argument Initialize a boolean flag `bool match_found = false;` for the current argument.

Step 2: Check the Current Working Directory (CWD) The spec explicitly mandates checking the CWD first.

1. Get the current absolute path using `getcwd()`.
2. Use `sprintf()` to safely concatenate: `cwd + "/" + filename`. (e.g., `/home/student/project/mycmd`).
3. Call `access(constructed_path, X_OK)`.
4. If it returns 0 (success), print the `constructed_path` and set `match_found = true;`.

Step 3: Iterate through PATH

1. Loop through your pre-computed `path_dirs` array.
2. For each directory, use `sprintf()` to concatenate: `dir + "/" + filename`.
3. Call `access(constructed_path, X_OK)`.
4. If it succeeds:
 - *Wait! Is the path absolute?* The spec requires printing the absolute path. While most PATH entries are absolute (like `/usr/bin`), someone could technically put `.` or `../bin` in their PATH. To be perfectly compliant, wrap the `constructed_path` in `realpath()` to resolve any relative segments before printing it.
 - Print the resolved path.
 - Set `match_found = true;`.

5. **Do not break out of the loop.** The spec states: *"If multiple matching executable paths are found, print all the paths."*

Step 4: Error Handling After checking the CWD and the entire PATH array, look at `match_found`.

- If it is still false, print exactly: `locate: command not found (filename)`.
- Continue to the next argument in the loop (do not abort the whole `locate` command).

4. Edge Cases & Pitfalls to Watch Out For

1. **The "Current Directory is in PATH" Duplication:** If a user has `.` (current directory) or an explicit path to the current directory inside their PATH variable, you might accidentally print the CWD match twice—once during Step 2, and once during Step 3.
 - *Fix:* When you find a match in Step 3, you can use `stat()` to compare the `st_ino` (inode number) and `st_dev` (device ID) of the match against the one you found in Step 2. If they match, it's the exact same file; skip printing it a second time.
2. **Buffer Overflows with `sprintf`:** When combining `dir`, `/`, and `filename`, make sure your buffer is dynamically allocated or securely sized to `PATH_MAX`. Check the return value of `sprintf` to ensure the path wasn't truncated before passing it to `access()`.
3. **Memory Leaks:** You `strdup'd` the PATH string, and you might have allocated an array for `path_dirs`. You also might use `realpath()` (which `mallocs` memory if you pass NULL as the second argument). Make sure every single one of these is `free()`d before the `locate` function returns.
4. **Empty PATH:** If `getenv("PATH")` returns NULL or an empty string, your shell shouldn't crash. It should just gracefully skip Step 3 and only check the CWD.

User prompt: C1

Response: Part C1 ("Command Execution") is where your shell finally transforms from a text parser into a true operating system interface. This is the heart of the UNIX execution model: the **Fork-Exec-Wait** paradigm.

Unlike Parts B1-B4, you are no longer simulating commands within the shell's own process space. You will be asking the OS kernel to spawn a brand new process and replace its memory with an external executable.

Here is the architectural blueprint for your execution engine.

1. Required POSIX System Calls & Headers

- **Process Creation:** `fork` (Header: `<unistd.h>`).
- **Process Execution:** `execv`, `execvp` (Header: `<unistd.h>`).
 - Note: `execvp` automatically searches the `$PATH` variable for you, while `execv` requires an exact path.
- **Process Synchronization:** `waitpid` (Header: `<sys/wait.h>`).
- **File Checking:** `access` (Header: `<unistd.h>`) with the `X_OK` flag.
- **Process Termination:** `exit` (Header: `<stdlib.h>`).

2. Phase 1: The Resolution Engine (What are we running?)

Before you fork, you must determine exactly *which* binary to run based on the strict rules in the specification. You can encapsulate this in a helper function: `char* resolve_command(char *cmd_name)`.

Architectural Logic: Look at the first character of the command (`Command.args[0]`):

1. Rule 1: Contains a / (Literal Path)

- Use `strchr(cmd_name, '/')` to check for this.
- If found, do not search. Just use `access(cmd_name, X_OK)` to verify it exists and is executable.
- If valid, `target = cmd_name`. If not, `target = NULL`.

2. Rule 2: Prefix % (Strict PATH search)

- If `cmd_name[0] == '%'`, strip the % (e.g., using `cmd_name + 1` or shifting the string left).
- You must skip the CWD check. Because you want to search PATH, you can simply rely on `execvp()` later, so `target = stripped_cmd_name`.

3. Rule 3: Standard Command (CWD first, then PATH)

- This is the default case. Standard Linux shells search PATH directly. **Your spec is different:** it mandates checking the Current Working Directory *first*.
- Construct a path to the CWD: `snprintf(buffer, sizeof(buffer), "%s", cmd_name)`.
- Check `access(buffer, X_OK)`.
- If it succeeds, `target = buffer` (which acts as a literal path).
- If it fails, `target = cmd_name` (meaning we will rely on `execvp` to search PATH).

If your resolution logic decides the command fundamentally doesn't exist (e.g., an absolute path failed access), you should print `"cshell: command not found (commandname)"` and abort early so you don't waste system resources forking.

3. Phase 2: The Fork-Exec-Wait Pattern

Once you know what to run, it is time to interface with the kernel.

Architectural Logic:

```
// 1. Create a child process
```

```

pid_t pid = fork();

if (pid < 0) {
    // fork failed (e.g., system out of memory)
    perror("fork failed");
}
else if (pid == 0) {
    // -----
    // CHILD PROCESS BLOCK
    // -----
    // We are now in the child process.

    // Call execv if we have a literal path (or resolved CWD path),
    // OR call execvp if we are relying on a PATH search.

    if (is_literal_path) {
        execv(resolved_path, command.args);
    } else {
        execvp(resolved_path, command.args);
    }

    // CRITICAL: If exec succeeds, it NEVER returns here.
    // If we reach this line of code, exec failed!
    print("cshell: command not found (%s)\n", original_cmd_name);

    // We MUST exit. If we don't, this child process will continue
    // running the shell REPL, resulting in two shells taking input!
    exit(EXIT_FAILURE);
}
else {
    // -----
    // PARENT PROCESS BLOCK
    // -----
    // We are in the parent (your shell).
    // We must wait for the specific child we just created.

    int status;
    waitpid(pid, &status, 0);

    // The child has finished. The parent continues the REPL loop.
}

```

4. Edge Cases & Pitfalls to Watch Out For

- 1. The "Fork Bomb" Bug (Missing exit):** If `execvp` fails (for example, the user typed a gibberish command like `dosakdaoskdos`), `execvp` returns `-1`. If you forget to put `exit(EXIT_FAILURE)` immediately after `execvp`, the child process will just keep executing the rest of your C code. Suddenly, you have *two* shells running in the same terminal. If the user types another bad command, you'll have four. This will quickly crash your machine.
- 2. exec Array Null-Termination:** The second argument to `execv` and `execvp` is an array of strings (`char **argv`). The kernel *requires* that the very last element of this array is `NULL`. If your parser in Part A3 did not append a `NULL` pointer to the end of `Command.args`, `exec` will read past the end of the array into garbage memory and likely segfault. (e.g., `["echo", "hello", NULL]`).
- 3. Built-ins vs. Executables:** In your `executor.c`, you must intercept built-in commands (`hop`, `reveal`, `peek`, `locate`, `activities`, `ping`, `spy`, `snoop`) **before** you fork.
 - If the command is `hop`, run your `hop` logic directly in the parent and return. Do not fork!
 - Only pass the command to the Fork-Exec-Wait engine if it is *not* one of your built-ins.
- 4. Mutating the Command Name for %:** If you modify the string to strip the `%` prefix (e.g., changing `%build.sh` to `build.sh`), remember to use the *original* name (`%build.sh`) if `execvp` fails and you need to print the error message `"cshell: command not found (%build.sh)"`. Keep a backup pointer to the original string.

User prompt: C2

Response: Part C2 ("Input Redirection") introduces a fascinating curveball. In standard shells like Bash, running `command < a.txt < b.txt` simply discards `a.txt` and only reads from `b.txt`. **Your specification demands something much more complex:** you must

concatenate multiple input files into a single, continuous stream.

To achieve this without blocking your main shell process, you will need to map this requirement onto the UNIX concept of pipes. Conceptually, your shell must quietly translate `command < f1 < f2` into `cat f1 f2 | command` under the hood.

Here is the architectural blueprint for your input redirection engine.

1. Required POSIX System Calls & Headers

You will build upon the syscalls from Part C1, adding the following specifically for redirection and stream merging:

- **File Opening:** `open` (Header: `<fcntl.h>`) with the `O_RDONLY` flag.
- **File Descriptor Duplication:** `dup2` (Header: `<unistd.h>`). This is the magic function that overwrites `STDIN_FILENO`.
- **Inter-Process Communication:** `pipe` (Header: `<unistd.h>`). Required to stream multiple files continuously into a single file descriptor.
- **I/O Operations:** `read`, `write`, `close` (Header: `<unistd.h>`).

2. High-Level Architecture & Data Structures

Your parser (from Part A3) should already be populating an array of input files inside your `Command` struct:

```
typedef struct {
    char **args;
    // ...
    char **input_files; // Array of strings: {"a.txt", "b.txt", NULL}
    int num_input_files; // e.g., 2
} Command;
```

Architecturally, this logic sits precisely **between** the `fork()` and the `execvp()` in your execution engine. However, because of the multi-file requirement, the parent process must validate the files *before* forking the main command.

3. Component Breakdown & Logical Flow

Here is the step-by-step logic to handle the three different scenarios (zero files, one file, and multiple files).

Step 1: Pre-flight Validation (In the Parent)

Before doing any process creation, you must ensure all requested input files exist and are readable.

1. Loop through `Command.input_files`.
2. Attempt to `open(file, O_RDONLY)`.
3. If the `open` fails (returns `-1`), immediately print `"cshell: no such file or directory"`.
4. **CRITICAL:** If any file fails, abort the execution of this command entirely. Do not fork. Do not run the command.
5. If the file opens successfully, `close()` it immediately. You are just checking permissions here.

Step 2: The Single File Fast-Path (Inside the Child)

If `num_input_files == 1`, the logic inside your forked child (before `execvp`) is simple:

1. `int fd = open(input_files[0], O_RDONLY);`
2. `dup2(fd, STDIN_FILENO);`
3. `close(fd);` (You must close the original fd since `STDIN` now holds a copy of it).
4. Call `execvp`.

Step 3: The Multi-File Stream Merger (The OS Challenge)

If `num_input_files > 1`, you cannot simply `dup2` them all. A process only has one Standard Input. To feed multiple files as a continuous stream, you must create a pipe and spawn a **Helper Process** that pumps data into that pipe.

Pseudocode for the Multi-File Flow:

PARENT PROCESS:

1. Create a pipe: `int pfd[2]; pipe(pfd);`
2. Fork Helper Process (The Stream Merger).
3. Fork Command Process (The Executor).

HELPER PROCESS (Child 1):

1. Close the read end: `close(pfd[0]);`
2. FOR EACH file in `input_files`:
`int fd = open(file, O_RDONLY);`
`while (bytes_read = read(fd, buffer, chunk_size)) > 0:`
`write(pfd[1], buffer, bytes_read);`
`close(fd);`
3. Close the write end: `close(pfd[1]);`
4. Exit successfully. (`exit(0)`)

COMMAND PROCESS (Child 2):

1. Close the write end: `close(pfd[1]);`
2. Redirect stdin: `dup2(pfd[0], STDIN_FILENO);`
3. Close the read end: `close(pfd[0]);`
4. Call `execvp`!

PARENT PROCESS (Cleanup):

1. Close BOTH ends of the pipe: `close(pfd[0]); close(pfd[1]);`
2. `waitpid()` for BOTH the Helper Process and the Command Process.

4. Edge Cases & Pitfalls to Watch Out For

1. **The Deadlock Trap (Blocking Pipes):** If you try to avoid the "Helper Process" by having the *parent* shell read the files and write them into the pipe, your shell will deadlock on large files. A UNIX pipe buffer is typically 64KB. If the files contain 100KB of text, the parent's `write()` will block, waiting for the child to read. But if the child hasn't been exec'd yet (or is waiting for STDIN), the whole shell freezes. **You must use a separate concurrent process (the Helper) to pump the data.**
 2. **File Descriptor Leaks:** Every time you call `pipe()` or `open()`, the OS allocates file descriptors. `dup2(oldfd, newfd)` copies `oldfd` onto `newfd`. That means *both* descriptors now point to the same file. If you don't `close(oldfd)`, it stays open forever. Run out of FDs, and your shell will crash.
 3. **Parent Pipe Closure:** In the multi-file scenario, the parent process *must* close both `pfd[0]` and `pfd[1]` after forking the two children. If the parent leaves the write end (`pfd[1]`) open, the Command Process will never receive an EOF (End of File) signal on its Standard Input, because the OS thinks the parent might still write to it. The command will hang indefinitely.
 4. **Error Handling After Forking:** If `dup2` fails inside the child, or if `execvp` fails, ensure you still call `exit(EXIT_FAILURE)`. Do not let a child process return to the REPL loop!
-

User prompt: C3

Response: Part C3 ("Output Redirection") introduces another brilliant twist. In a standard shell like Bash, if you type `echo hi > a.txt > b.txt`, the shell usually discards `a.txt` and only writes to `b.txt` (or leaves `a.txt` empty).

Your specification demands a broadcast (tee) behavior: "Every listed file must receive the complete output of the command."

Since a process only has a single Standard Output (`STDOUT_FILENO`), you cannot simply `dup2` multiple files onto it. You must architect a "Broadcaster" system. Here is the blueprint.

1. Required POSIX System Calls & Headers

- **File Opening:** `open` (Header: `<fcntl.h>`). You will specifically need the flags:
 - `O_WRONLY`: Open for writing.
 - `O_CREAT`: Create the file if it doesn't exist.
 - `O_TRUNC`: Erase the file contents before writing (for `>`).
 - `O_APPEND`: Append to the end of the file (for `>>`).

- **File Modes:** 0644 (rw-r--r--). You *must* pass this as the third argument to `open()` whenever you use `O_CREAT`.
- **Process & IPC:** fork, pipe, dup2, waitpid.

2. High-Level Architecture & Data Structures

Your parser should populate the `Command` struct to track both the filenames and their respective modes (Truncate vs. Append):

```
typedef struct {
    char **args;
    // ...
    char **output_files; // Array: {"out1.txt", "out2.txt", NULL}
    int *append_modes;    // Array: {0, 1} (0 means '>', 1 means '>>')
    int num_output_files; // e.g., 2
} Command;
```

3. Component Breakdown & Logical Flow

Step 1: Pre-flight Validation (The Permission Check)

Before forking the main command, you must ensure that *all* listed files can be created or written to.

1. In the parent shell, loop through `Command.output_files`.
2. Determine the flags: `int flags = O_WRONLY | O_CREAT | (append_modes[i] ? O_APPEND : O_TRUNC);`
3. Attempt to open: `int fd = open(output_files[i], flags, 0644);`
4. If `fd == -1`, immediately print "cshell: unable to create file for writing", close any previously opened files, and **abort the command entirely**.
5. If you are going the Helper Process route (below), you can keep these file descriptors open in an array to pass to the helper. Otherwise, close them for now.

Step 2: Single File Fast-Path (Inside the Command Child)

If `num_output_files == 1`, you don't need a complex broadcaster. Inside your forked Command Process (before `execvp`):

1. `int fd = open(output_files[0], flags, 0644);`
2. `dup2(fd, STDOUT_FILENO);`
3. `close(fd);`
4. Execute the command.

Step 3: The Broadcaster Pattern (For Multiple Files)

If `num_output_files > 1`, you must intercept the command's standard output and manually duplicate it to every file. You do this by creating a pipe and a **Helper Process** (similar to the UNIX `tee` command).

Pseudocode for the Multi-File Flow:

PARENT PROCESS:

1. Create a pipe: `int out_pfd[2]; pipe(out_pfd);`
2. Fork Helper Process (The Broadcaster).
3. Fork Command Process (The Executor).

HELPER PROCESS (The Broadcaster):

1. Close the write end of the pipe: `close(out_pfd[1]);`
2. Open ALL output files (keeping their FDs in an array).
3. `while (bytes_read = read(out_pfd[0], buffer, chunk_size)) > 0:`
 FOR EACH `fd` in `opened_files`:
 `write(fd, buffer, bytes_read);`
4. Close all file descriptors.
5. Exit successfully. (`exit(0)`)

COMMAND PROCESS (The Executor):

1. Close the read end of the pipe: `close(out_pfd[0]);`
2. Redirect stdout into the pipe: `dup2(out_pfd[1], STDOUT_FILENO);`

3. Close the write end: `close(out_pfd[1]);`
4. Call `execvp!`

PARENT PROCESS (Cleanup):

1. Close BOTH ends of the pipe!
2. `waitpid()` for both the Helper Process and the Command Process.

4. Edge Cases & Pitfalls to Watch Out For

1. **The Missing 0644 Argument:** When calling `open()` with `O_CREAT`, you **must** supply the third argument (the mode): `open(path, flags, 0644)`. If you omit this, C will read garbage from the stack as the file permissions, resulting in files you can't read, write, or delete (e.g., a file with `-----` or `?x?x?x?x` permissions).
2. **Combining Input (C2) and Output (C3):** The spec states: *"Input and output redirection must work together"*. If a command has both, your Command Process just does *two* `dup2` calls before `execvp`:
 - `dup2(in_pfd[0], STDIN_FILENO);` (or the direct file FD)
 - `dup2(out_pfd[1], STDOUT_FILENO);` (or the direct file FD) The order of these two `dup2` calls does not matter.
3. **Terminal Output Leakage:** If you mess up your `dup2` or forget to close the original file descriptors, the command might still print to the terminal. The spec strictly requires that when `>` is present, output *must not* appear on the terminal. Ensure you completely overwrite `STDOUT_FILENO` (file descriptor 1).
4. **Zombie Helper Processes:** Whenever you fork a Helper Process (for C2 or C3), the parent shell must eventually `waitpid()` on it. If you only `waitpid()` on the main command and forget the helpers, they become "zombies" taking up process table slots. You can either wait for them explicitly, or set them up to be automatically reaped (though explicit waiting is much safer for Part C).

User prompt: C4

Response: Part C4 ("Command Piping") is the crowning achievement of any custom shell. It marries process creation (C1) with IPC file descriptor manipulation (C2 & C3) to implement the UNIX philosophy: writing small, modular programs that chain together perfectly.

Managing pipes is notoriously prone to deadlocks. If you leave even a single write-end of a pipe open in the wrong process, your pipeline will hang indefinitely.

Here is the architectural blueprint for your iterative pipeline engine.

1. Required POSIX System Calls & Headers

You are combining everything you have learned so far:

- **Inter-Process Communication:** `pipe` (Header: `<unistd.h>`).
- **Process Control:** `fork`, `waitpid` (Header: `<sys/wait.h>`).
- **Descriptor Routing:** `dup2`, `close` (Header: `<unistd.h>`).
- **Execution:** `execvp` or `execv`.

2. High-Level Architecture: The Iterative Engine

A pipeline is a sequence of N commands connected by $N - 1$ pipes. Do not try to write hardcoded logic for "2 commands" or "3 commands". You must build a `for` loop that can handle a pipeline of arbitrary length.

To do this, the parent shell must carry the "read end" of the previous iteration's pipe forward into the next iteration.

Conceptual Plumbing:

```

      PIPE 0          PIPE 1
CMD 0 -----> CMD 1 -----> CMD 2
      write(p0[1])    read(p0[0])    read(p1[0])
                    write(p1[1])

```

3. Component Breakdown & Logical Flow

Here is the step-by-step logic for your pipeline execution loop.

Step 1: Setup

1. Determine `num_commands` in the Pipeline.
2. Create an array of `pid_t pids[num_commands]` so the parent can wait for all children later.
3. Initialize an integer tracking the input from the previous pipe: `int prev_pipe_read = -1;` (or you can default it to `STDIN_FILENO`).

Step 2: The Pipeline Loop Iterate `i` from 0 to `num_commands - 1`:

1. Create the Next Pipe (if needed):

- If `i < num_commands - 1` (i.e., this is NOT the last command), call `pipe(current_pipe)`.

2. Fork the Child:

- `pids[i] = fork();`

3. In the Child Process:

- *Input Wiring*: If there is a previous pipe (`prev_pipe_read != -1`), `dup2(prev_pipe_read, STDIN_FILENO)` and then `close(prev_pipe_read)`.
- *Output Wiring*: If there is a next pipe (not the last command), `dup2(current_pipe[1], STDOUT_FILENO)`.
- *Cleanup*: The child must `close(current_pipe[0])` (it doesn't read from its own output pipe) and `close(current_pipe[1])` (it just duplicated it to `STDOUT`).
- *Redirection Override*: Apply any C2/C3 file redirections **NOW**. (See Section 4 below).
- *Execute*: Call `execvp`. If it fails, print `"cshell: command not found"` and `exit(EXIT_FAILURE)`.

4. In the Parent Process:

- *Close the Old*: If `prev_pipe_read != -1`, `close(prev_pipe_read)`. The parent doesn't need the previous pipe's read end anymore; it just gave it to child `i`.
- *Close the Current Write End*: If not the last command, **CRITICAL**: `close(current_pipe[1])`. The parent will never write to this pipe.
- *Save the Read End for Tomorrow*: If not the last command, `prev_pipe_read = current_pipe[0]`.

Step 3: Synchronization (The Wait Loop) After the `for` loop finishes launching all children, the parent must wait for them all to complete.

- Loop `i` from 0 to `num_commands - 1`.
- Call `waitpid(pids[i], &status, 0);`.

4. Intersections: Pipes + File Redirection

The specification strictly notes: *"File redirection and pipes must work together (e.g. `command1 < input.txt | command2 > output.txt`)."*

This sounds complicated, but it is actually incredibly elegant if you layer it correctly in the child process.

The Golden Rule of Routing: Pipe wiring goes first, File wiring goes second.

Inside the child process:

1. **Wire the pipes** using `dup2` as described in Step 2 above. (e.g., `STDOUT` points to the pipe).
2. **Evaluate Redirections**: Check if `Command.output_files` or `Command.input_files` exist for this specific command in the pipeline.
3. If a file redirection exists, execute the C2/C3 logic (open the file, `dup2` it to `STDIN/STDOUT`).
 - *Why this works*: If you `dup2` a file to `STDOUT_FILENO`, it silently closes the pipe connection that was previously sitting on `STDOUT` and replaces it with the file. The next command in the pipeline will simply read an immediate EOF (0 bytes) from the pipe, which is exactly the expected POSIX behavior!

5. Edge Cases & Pitfalls to Watch Out For

1. **The "Dangling Pipe" Deadlock:** This is the #1 reason shell projects fail. `read()` on a pipe blocks until there is data OR until all write file descriptors pointing to that pipe are closed (which returns EOF). If the parent shell forgets to `close(current_pipe[1])` during the loop, child `i+1` will hang forever waiting for an EOF that will never arrive, because the OS sees that the parent shell *might* theoretically write to that pipe someday.
2. **FD Exhaustion in the Parent:** If you have a pipeline of 10 commands, you generate 9 pipes (18 file descriptors). If the parent doesn't close them on every iteration, they accumulate. Always close `prev_pipe_read` in the parent once it has been handed off.
3. **Command Failures within the Chain:** The spec states: *"If a command in the pipeline fails... the rest of the pipeline still runs."* For example: `echo hi | badcmd | wc -c`.
 - `echo` runs.
 - `badcmd` fails `execvp`, prints the error, and calls `exit(EXIT_FAILURE)`.
 - Because `badcmd` exits, the OS automatically closes its write-end of the pipe.
 - `wc` reads an EOF from the pipe, counts 0 bytes, and prints 0.
 - If you built your pipeline correctly, this fallback behavior happens entirely automatically!
4. **Waiting for the Wrong Things:** Make sure you wait for *every* PID in the pipeline. Some implementations only `waitpid()` on the very last command. While the terminal will return to the prompt correctly, the preceding commands will turn into Zombie processes (defunct) taking up memory until the shell closes. Wait for all of them.

User prompt: a bit more detailed

Response: Let's put the File Descriptor (FD) table under a microscope. When students struggle with C4, it is almost always because they lose track of which file descriptors are open in which process.

To give you the rigorous architectural breakdown you need, we must look at how the OS kernel manages reference counts on pipe endpoints. A pipe is just a kernel memory buffer with two ends: read and write. **The kernel will only send an EOF to the read end if the reference count of the write end drops to exactly zero across ALL processes.**

Here is a microscopic trace of a three-command pipeline: `cat | sort | uniq`

1. The Setup (Before the Loop)

Every process starts with three default FDs:

- 0: STDIN (Terminal Read)
- 1: STDOUT (Terminal Write)
- 2: STDERR (Terminal Write)

In the parent shell, you initialize a tracking variable: `int prev_pipe_read = -1;`

2. Iteration 0: cat (The First Command)

Parent Action: Because this is not the last command, the parent creates a pipe. Let's assume `pipe(fd)` returns 3 (read) and 4 (write). The parent forks Child 0.

Inside Child 0 (cat):

1. **Input:** `prev_pipe_read` is -1. No previous pipe exists. STDIN remains 0 (Terminal).
2. **Output:** It needs to output to Pipe 1.
 - `dup2(4, 1);` -> STDOUT (1) is now a clone of FD 4. Both 1 and 4 point to the pipe's write end.
3. **Cleanup:** Child 0 closes 3 and 4.
 - *Why?* It doesn't need to read from Pipe 1 (closes 3). It already has a copy of the write end on FD 1 (closes 4).

4. **Execute:** `execvp("cat", ...)`

Inside Parent (After Fork):

1. The parent closes 4 (the write end). The parent will *never* write to this pipe. If it leaves 4 open, the next command (sort) will hang forever waiting for EOF.
2. The parent saves 3 for the next iteration: `prev_pipe_read = 3;`

3. Iteration 1: sort (The Middle Command)

Parent Action: Not the last command, so the parent creates Pipe 2. Let's assume it gets FDs 4 (read) and 5 (write). The parent forks Child 1.

Inside Child 1 (sort):

1. **Input:** It must read from Pipe 1 (stored in `prev_pipe_read`, which is 3).
 - `dup2(3, 0);` -> STDIN (0) now points to Pipe 1's read end.
2. **Output:** It must write to Pipe 2 (FD 5).
 - `dup2(5, 1);` -> STDOUT (1) now points to Pipe 2's write end.
3. **Cleanup (CRITICAL ZONE):**
 - Close `prev_pipe_read` (3).
 - Close Pipe 2's read end (4).
 - Close Pipe 2's write end (5).
4. **Execute:** `execvp("sort", ...)`

Inside Parent (After Fork):

1. Close `prev_pipe_read` (3). The parent no longer needs to hold Pipe 1's read end because it just handed it off to Child 1.
2. Close Pipe 2's write end (5).
3. Save Pipe 2's read end: `prev_pipe_read = 4;`

4. Iteration 2: uniq (The Last Command)

Parent Action: This is the last command. **Do not create a new pipe.** The parent simply forks Child 2.

Inside Child 2 (uniq):

1. **Input:** It must read from Pipe 2 (`prev_pipe_read`, which is 4).
 - `dup2(4, 0);` -> STDIN (0) now points to Pipe 2's read end.
2. **Output:** No next pipe exists. Do nothing. STDOUT remains 1 (Terminal).
3. **Cleanup:**
 - Close `prev_pipe_read` (4).
4. **Execute:** `execvp("uniq", ...)`

Inside Parent (After Fork):

1. Close `prev_pipe_read` (4).

5. The Wait Array Architecture

If you call `waitpid` inside the loop right after forking, your pipeline becomes **sequential**, not parallel. `cat` would run to completion, filling up the 64KB pipe buffer and blocking, while `sort` hasn't even been spawned yet. Deadlock.

The correct synchronization structure:

```

Initialize pid_t pids[num_commands];

FOR i = 0 to num_commands - 1:
    // Pipe creation logic...
    pids[i] = fork();

    IF child:
        // dup2 and close logic...
        // execvp

    IF parent:
        // close unused pipe ends...
        // update prev_pipe_read...

// OUTSIDE THE LOOP
FOR i = 0 to num_commands - 1:
    waitpid(pids[i], &status, 0);

```

This guarantees all N processes are alive in the OS scheduler simultaneously, reading and writing to each other in real-time.

6. File Redirection Override in a Pipeline

Let's look at the Golden Rule of Routing with a concrete example: `cat < in.txt | sort > out.txt`.

Command 0 (cat < in.txt):

1. **Pipe wiring:** Child 0 duplicates Pipe 1's write end to `STDOUT_FILENO`.
2. **File wiring:** The parser sees `< in.txt`. Child 0 opens `in.txt` (gets FD 6). It duplicates FD 6 to `STDIN_FILENO`.
3. **Result:** `cat` reads from `in.txt`, and its output naturally flows into the pipe.

Command 1 (sort > out.txt):

1. **Pipe wiring:** Child 1 duplicates Pipe 1's read end to `STDIN_FILENO`.
2. **File wiring:** The parser sees `> out.txt`. Child 1 opens `out.txt` (gets FD 6). It duplicates FD 6 to `STDOUT_FILENO`.
 - *Notice what happens here:* Step 1 pointed `STDOUT_FILENO` to the terminal (since it's the last command). Step 2 immediately overwrites it. `STDOUT_FILENO` now points to the file.
3. **Result:** `sort` reads from the pipe, and writes securely to `out.txt`.

By strictly enforcing the rule—**Pipe wiring first, File wiring second**—your child processes will elegantly overwrite the pipe architecture whenever explicit file redirections are requested by the user.

User prompt: prepare a makefile for all

Response: While I won't write your C logic for you, a Makefile is part of the build infrastructure, and writing a clean, modular one is an essential systems programming skill.

Based on your project spec, your repository is structured like this:

```

c-shell/
├── src/           (Where your .c files live)
├── include/       (Where your .h files live)
└── Makefile       (Right here)

```

The spec also strictly mandates a specific set of compilation flags and requires that `make all` produces an executable named `shell.out` in the root `c-shell` directory.

Here is the architectural template for a professional, modular Makefile that automatically detects all your `.c` files so you never have to update it when you add a new component.

The Modular Makefile

```

# 1. Compiler and Flags
CC = gcc

```

```

# The exact flags mandated by your spec, plus -I to tell gcc where your headers are
CFLAGS = -std=c23 \
        -D_POSIX_C_SOURCE=200809L \
        -D_XOPEN_SOURCE=700 \
        -Wall -Wextra -Werror \
        -Wno-unused-parameter \
        -fno-asm \
        -I$(INC_DIR)

# 2. Directories
SRC_DIR = src
INC_DIR = include
OBJ_DIR = obj

# 3. Files
# Automatically find all .c files in the src/ directory
SRCS = $(wildcard $(SRC_DIR)/*.c)

# Translate the list of .c files into a list of .o files in the obj/ directory
OBJS = $(SRCS:$(SRC_DIR)/%.c=$(OBJ_DIR)/%.o)

# The final executable name mandated by the spec
TARGET = shell.out

# 4. Phony Targets (targets that aren't actual files)
.PHONY: all clean

# 5. Build Rules

# Default target required by spec
all: $(TARGET)

# Link all object files into the final executable
$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) $^ -o $@

# Compile each .c file into a .o file
# The '| $(OBJ_DIR)' is an order-only prerequisite ensuring the obj dir exists
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c | $(OBJ_DIR)
    $(CC) $(CFLAGS) -c $< -o $@

# Create the object directory if it doesn't exist
$(OBJ_DIR):
    mkdir -p $(OBJ_DIR)

# Clean up build artifacts
clean:
    rm -rf $(OBJ_DIR) $(TARGET)

```

Architectural Breakdown (How it works)

Makefiles look cryptic, but they operate on a simple dependency graph: **Target -> Prerequisites -> Command**.

- 1. Automatic File Discovery (wildcard & Substitution):** Instead of manually typing `src/main.c src/parser.c src/executor.c`, the `$(wildcard ...)` function scans the folder for you. The substitution rule `$(SRCS:$(SRC_DIR)/%.c=$(OBJ_DIR)/%.o)` magically transforms that list into `obj/main.o obj/parser.o obj/executor.o`.
- 2. The -I\$(INC_DIR) Flag:** This is critical. By passing `-Iinclude/` to gcc, you can write `#include "parser.h"` in your C files instead of `#include "../include/parser.h"`. It keeps your includes clean.
- 3. Automatic Variables (The cryptic symbols):**
 - `$@`: Means "The name of the target being generated".
 - `$<`: Means "The first prerequisite" (the specific .c file being compiled).
 - `$^`: Means "All prerequisites" (all the .o files to link together).
- 4. The obj/ Directory Isolation:** Notice that I added an `obj/` directory. If you don't do this, your `src/` folder will be littered with .o files, making it incredibly messy to read. This Makefile cleanly routes all compiled object files into `obj/` and only places the final `shell.out` in your root folder.

⚠ The Makefile Golden Rule

If you copy-paste this, you **must** ensure that the indented lines under the targets (the lines starting with `$(CC)`, `mkdir`, and `rm`) use a **literal TAB character**, not spaces. Make is infamous for crashing with "missing separator" errors if you use spaces for indentation.

To use it:

- Run `make` or `make all` to build everything.
- Run `make clean` before submitting to clear out binaries.

User prompt: D

Response: Part D introduces asynchronous execution and signal handling. Up to this point, your shell has been entirely synchronous: it asks the OS to do something, and it waits until it's done.

Now, you are building a system where multiple things happen at once. Background processes (`&`) will finish at unpredictable times, and the OS will notify your shell via software interrupts (Signals).

Here is your architectural blueprint for orchestrating Sequential and Background execution.

1. Required POSIX System Calls & Headers

- **Signal Handling:** `sigaction`, `sigemptyset` (Header: `<signal.h>`).
 - *Mentor Note:* Do not use the basic `signal()` function. It behaves inconsistently across different UNIX systems. `sigaction` is the POSIX standard for robust signal handling.
- **Asynchronous Waiting:** `waitpid` with the `WNOHANG` flag (Header: `<sys/wait.h>`).
- **Process Groups (The Part E foreshadowing):** `setpgid` (Header: `<unistd.h>`).

2. High-Level Architecture & Data Structures

You need to upgrade two parts of your shell: the AST (Abstract Syntax Tree) to handle a list of pipelines, and a global Job Registry to track background tasks.

1. The Command Sequence (AST Upgrade): A single line of input can now contain multiple pipelines separated by `;` or `&`.

```
typedef enum { SEP_NONE, SEP_SEMI, SEP_AMP } Separator;

// A sequence is an array of Pipelines
typedef struct {
    Pipeline *pipelines;
    Separator *separators; // What comes AFTER this pipeline (; or &)
    int num_pipelines;
} CommandSequence;
```

2. The Job Registry (Global State): Because signal handlers interrupt your code at random times, they need a global place to look up and modify running jobs.

```
typedef enum { JOB_RUNNING, JOB_DONE_NORMAL, JOB_DONE_ABNORMAL } JobStatus;

typedef struct {
    int job_number;           // Monotonically increasing (1, 2, 3...)
    pid_t pgid;              // Process Group ID (PID of the first command in the pipeline)
    char *command_string;    // The original text (e.g., "sleep 10 &")
    JobStatus status;        // Updated by the SIGCHLD handler
} BackgroundJob;

// Global state
BackgroundJob job_list[MAX_JOBS];
int global_job_counter = 1;
```

3. Component Breakdown & Logical Flow

D1: Sequential Execution (;)

This is surprisingly straightforward now that your Part C engine works.

1. Iterate through your `CommandSequence.pipelines`.
2. For each pipeline, run your Part C execution engine (fork, wire pipes, exec, waitpid).
3. **The Failure Hook:** The spec requires that if a command fails to execute, you stop the remaining sequence.
 - *How does the parent know execvp failed?* In Part C, if `execvp` fails, you print an error and call `exit(EXIT_FAILURE)`. Change that to a specific, unique exit code (e.g., `exit(127)`).
 - When the parent's `waitpid` returns, use the `WIFEXITED(status)` and `WEXITSTATUS(status)` macros. If the exit status is 127, break out of your sequence loop immediately.

D2: Background Execution (&) and Process Groups

If the separator is `&`, you run the exact same Part C execution engine, but with two major changes:

1. **Do not call `waitpid`** in the parent shell for this pipeline.
2. **Assign a Process Group:** As hinted by the spec's note, you must put this entire pipeline into its own Process Group.
 - In the parent, after forking the *first* command of the pipeline, save its PID.
 - For every child in that pipeline, call `setpgid(child_pid, first_command_pid)` immediately after `fork()`.
 - By doing this, the background jobs detach from the terminal's input stream. If they try to read from `stdin`, the kernel will automatically stop them with a `SIGTTIN` signal, fulfilling the requirement: *"Background processes must not have access to the terminal for input."*
3. **Register the Job:** Add the job to your `job_list`, assigning it `global_job_counter++`.
4. **Print the Launch Message:** Print `[%d] %d\n` using the job number and the PID of the *first* command in the pipeline.

4. The Signal Architecture (SIGCHLD)

When a background process dies, the kernel sends a `SIGCHLD` signal to your shell.

The Trap of `printf` in Signal Handlers: The spec says you must report when a background process terminates, **BUT** it also says: *"If any foreground process is being executed... its completion must be reported after the foreground process terminates."*

This is actually a massive blessing. Calling `printf` inside a signal handler is strictly forbidden in C (it is not "async-signal-safe" and can deadlock your program if the signal interrupts an existing `printf`).

The Two-Part Reaping Architecture:

Part 1: The Handler (Fast & Silent) Use `sigaction` in your `main()` function to register a handler for `SIGCHLD` before your REPL loop starts.

```
FUNCTION handle_sigchld(int sig):
    // Save errno because waitpid might change it
    int saved_errno = errno;

    // Use a loop! Multiple children might die at the exact same time.
    // WNOHANG ensures we don't block if a child is still running.
    WHILE (pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0:
        Find this pid in the global job_list
        IF WIFEXITED(status):
            job.status = JOB_DONE_NORMAL
        ELSE IF WIFSIGNALED(status):
            job.status = JOB_DONE_ABNORMAL

    errno = saved_errno;
```

Part 2: The Printer (Safe & Synchronous) Since the handler just flips a state flag, you do the actual printing safely in your main thread at two specific locations:

1. **Before printing the A1 prompt:** Iterate through `job_list`. If any job is marked `DONE`, print its exit message and remove it from the list.

2. **After a foreground pipeline finishes:** Right after your synchronous `waitpid` loop from Part C completes, iterate through the `job_list` and print any DONE jobs.

This perfectly fulfills the requirement to defer printing if a foreground process is running, and guarantees thread-safety!

5. Edge Cases & Pitfalls to Watch Out For

1. **The EINTR Interruption (Crucial):** When a signal (like `SIGCHLD`) is caught, it interrupts whatever the main program was doing. If your shell was sitting at `getline()` waiting for user input, `getline` will suddenly fail and return `-1`, and the global `errno` will be set to `EINTR` (Interrupted System Call).
 - *Fix:* In your Part A2 REPL loop, if `getline` returns `-1`, you must check if `(errno == EINTR)`. If it is, `clearerr(stdin)` and continue the loop. Do not exit the shell!
2. **Race Conditions in `setpgid`:** When you fork a child, you don't know who runs first: the parent or the child. If the child tries to call `execvp` before it gets placed in its new process group, things can get messy.
 - *Fix:* Call `setpgid(pid, first_command_pid)` in **both** the parent (immediately after `fork`) and the child (immediately before `execvp`). The OS will ignore the redundant call, completely eliminating the race condition.
3. **Zombie Pipelines:** In a background pipeline `cmd1 | cmd2 &`, the `SIGCHLD` handler will reap both `cmd1` and `cmd2`. However, the printed exit message should only happen once for the whole job. Architect your `JobRegistry` so it tracks the process group, and only marks the job as DONE when the *last* process in that pipeline exits.

User prompt: *E*

Response: Part E ("Exotic Shell Intrinsic") transforms your shell from a basic command executor into a true job-controlling OS interface. You will be wrestling with Process Groups, Terminal Control (`tty`), and asynchronous signal multiplexing.

This is where standard C programming ends and hardcore UNIX systems programming begins.

Here is the architectural blueprint for conquering Part E.

1. The Core OS Concept: Process Groups and The Terminal

Right now, your shell and all its children share the terminal. If a user presses `Ctrl+C`, the OS terminal driver broadcasts a `SIGINT` to **every** process attached to the terminal. That would kill your shell!

To fix this, UNIX groups processes into **Process Groups (PGIDs)**. The terminal driver has a special pointer called the **Foreground Process Group**. When `Ctrl+C` or `Ctrl+Z` is pressed, the terminal *only* sends the signal to the Foreground Process Group.

Your shell's new job is to juggle which process group gets to be in the "Foreground" at any given time.

2. High-Level Architecture & Data Structure Upgrades

In Part D, you built a `JobRegistry`. To support E1 (activities), you must completely upgrade this data structure. You can no longer just track the "job"; you must track **every individual process** within a pipeline.

```
typedef enum { PROC_RUNNING, PROC_STOPPED } ProcState;

// Tracks a single process in a pipeline
typedef struct {
    pid_t pid;
    char *command_name; // e.g., "cat" or "sort"
    ProcState state;
    bool is_done;       // True if waitpid reaped it
} Process;

// Tracks the entire pipeline (Process Group)
typedef struct {
    int job_number;
    pid_t pgid;         // Process Group ID (PID of the first command)
    Process *procs;     // Array of processes in this pipeline
    int num_procs;
    char *full_command; // e.g., "cat | sort"
} Job;
```

```
// Global Registry
Job job_registry[MAX_JOBS];
```

The New Pipeline Lifecycle:

1. Before forking a pipeline, add a Job to the registry.
2. As you fork each child, add its PID to the Job.procs array.
3. If a process finishes (detected by waitpid in fg, or SIGCHLD in bg), mark is_done = true.
4. If *all* processes in a Job are is_done, remove the Job from the registry entirely.

3. Component Breakdown & Logical Flow

E1: activities (The Job Lister)

Because of your upgraded data structure, this becomes a simple printing function.

1. Iterate through job_registry.
2. For each job, count how many processes are !is_done. If the count is 0, skip the job.
3. Print the Group Line: [%d] pgid %d.
4. Iterate through job.procs. For every process that is !is_done, print its PID, command_name, and state (Running/Stopped).

E2: Terminal Control (The Danger Zone)

This requires surgical precision with POSIX APIs.

1. Startup (In main.c):

- Ignore signals so the shell doesn't kill itself: signal(SIGINT, SIG_IGN), signal(SIGTSTP, SIG_IGN).
- **CRITICAL:** You must also ignore SIGTTOU. If you don't, your shell will freeze the moment it tries to take the terminal back.

2. Foreground Execution (In your executor):

- Put the pipeline in its own PGID using setpgid().
- Give the terminal to the pipeline: tcsetpgrp(STDIN_FILENO, child_pgid).
- Wait for the pipeline: waitpid(-child_pgid, &status, WUNTRACED). (The - means wait for *any* process in that process group. WUNTRACED tells it to return if the child is stopped by Ctrl-Z).
- **Reclaim the terminal:** tcsetpgrp(STDIN_FILENO, shell_pgid).

3. The waitpid Evaluation:

- If WIFSTOPPED(status) is true, the user pressed Ctrl+Z. Iterate through the Job.procs and set their states to PROC_STOPPED. Print [%d] + Stopped. Leave the job in the registry.

E3: resume (Foreground and Background)

1. **Parse & Locate:** Parse the %job_number, look it up in your job_registry. If not found, print resume: no such job.
2. **Awaken the Group:** Send a continue signal to the entire process group: kill(-pgid, SIGCONT). Mark all processes in that job as PROC_RUNNING.
3. **If bg:** Print [%d] + Running ... and return to the prompt.
4. **If fg:**
 - tcsetpgrp to give them the terminal.
 - If --timeout <sec> is provided, call alarm(seconds).
 - Call waitpid just like you do for a normal foreground command.

- Call `alarm(0)` to cancel the timer once `waitpid` finishes.
- Reclaim the terminal with `tcsetpgrp`.

5. Timeout Handler (SIGALRM):

- If the alarm goes off, your `waitpid` will fail with `EINTR`.
- Catch this, send `kill(-pgid, SIGTERM)`, print "resume: job timed out", and reclaim the terminal.

E4: ping (The Signal Sender)

1. **Validation:** Check if `signal_number` is a valid integer ≥ 0 . The actual signal to send is `sig % 64`.
2. **Target Tracking Check:** The spec dictates you can *only* signal processes your shell tracks.
 - If target is `%X`: Look up job `X` in the registry. Send `kill(-pgid, sig)`.
 - If target is PID: Loop through every Job and every Process inside the registry. If you find the PID, it's valid! Send `kill(pid, sig)`. If you finish the loop without finding it, print `ping: no such process found`.

4. Edge Cases & Pitfalls to Watch Out For

1. **The Double-Ctrl-D Exit Logic:** When reading input with `getline`, if it returns EOF (Ctrl-D), you must check the `job_registry` for any jobs with `PROC_STOPPED`.
 - If stopped jobs exist, print "cshell: there are stopped jobs" and *do not exit*.
 - Set a boolean flag `warned = true`.
 - If the very next iteration is *also* an EOF, and `warned == true`, then you exit. If the user types a normal command in between, reset `warned = false`.
2. **The SIGHUP Nuke:** When the shell finally does exit (via Ctrl-D or `exit`), you must loop through the `job_registry`. For every job that still exists (running or stopped), send a Hangup signal: `kill(-job.pgid, SIGHUP)`. This ensures you don't leave orphaned background processes churning up CPU on the host machine.
3. **Terminal Race Conditions:** When setting up a foreground pipeline, the parent calls `tcsetpgrp()`. But what if the child tries to `execvp` and read from `stdin` before the parent gets scheduled to run? The child will crash. *Fix:* The child process must *also* call `setpgid()` and `tcsetpgrp()` immediately after `fork()` and before `execvp()`. Yes, it's redundant. Yes, POSIX requires it to avoid race conditions.
4. **SIGCHLD and tcsetpgrp Collision:** In your `SIGCHLD` handler (from Part D), you might be tempted to call `tcsetpgrp` if a background process dies. **Do not do this.** Background processes do not own the terminal. Only touch `tcsetpgrp` in your main synchronous execution loop. Let the signal handler quietly update the `is_done` flags.

User prompt: *F*

Response: Part F ("Fun Stuff") steps outside the boundaries of a traditional shell and dips your toes into the realm of system utilities and debuggers. You are effectively building mini-versions of the classic Linux commands `ls` of (List Open Files) and `strace` (System Call Tracer).

Unlike Parts A-E which rely on POSIX standards, Part F heavily leverages Linux-specific features: the `/proc` pseudo-filesystem and the `ptrace` API.

Here is your architectural blueprint for building these advanced diagnostic tools.

1. Required System Calls & Linux Headers

- **File System (F1):** `readlink` (Header: `<unistd.h>`), `stat` / `lstat` (Header: `<sys/stat.h>`).
- **Time Measurement (F2):** `clock_gettime` (Header: `<time.h>`).
- **Process Tracing (F2):** `ptrace` (Header: `<sys/ptrace.h>`), `<sys/user.h>` (for accessing CPU registers).
- **Synchronization (F2):** `waitpid` with `WIFSTOPPED` macros.

2. Component Breakdown: F1 (spy) - Listing Open Files

Linux exposes the internal state of the kernel through a virtual filesystem mounted at `/proc`. Everything you need to know about a process is represented as a file or directory inside `/proc/[pid]/`.

Step 1: Target Resolution

- If `Command.argc == 1`, the target PID is the shell's own PID (`getpid()`).
- If `Command.argc == 2`, parse the string into an integer.
- Use `access("/proc/[pid]", F_OK)` to verify the process exists. If it fails, print "spy: no such process".

Step 2: The Four Probes

To gather the required outputs (`cwd`, `txt`, `mem`, and numeric FDs), you must probe four specific locations in `/proc/[pid]/`.

Write a helper function: `void print_spy_entry(int pid, const char *fd_name, const char *link_path)`

1. Use `lstat(link_path, &st)` to determine the TYPE (e.g., `S_ISDIR -> DIR`, `S_ISREG -> REG`, `S_ISCHR -> CHR`).
2. Use `readlink(link_path, buffer, sizeof(buffer))` to get the actual PATH.
3. Print the formatted line.

Probe A: Current Working Directory (`cwd`)

- Target: `/proc/[pid]/cwd`
- Pass "cwd" as the `fd_name`.

Probe B: Executable Text (`txt`)

- Target: `/proc/[pid]/exe`
- Pass "txt" as the `fd_name`.

Probe C: Memory Mapped Files (`mem`)

- Target: Read the text file `/proc/[pid]/maps`.
- This file lists all memory segments. Read it line by line using `getline()`.
- Extract the file path from the end of each line (if it exists).
- *Requirement:* "Each unique memory mapped file should be printed only once." Maintain a linked list or hash set of strings. Before printing a `mem` entry, check if you've already seen that exact path.

Probe D: Numeric File Descriptors (0, 1, 2...)

- Target: Open the directory `/proc/[pid]/fd/` using `opendir()`.
- Iterate through the directory using `readdir()`.
- For every entry (which will be a number like 0, 1, 2, 3), pass that number as the `fd_name` and `/proc/[pid]/fd/[number]` as the `link_path`.

3. Component Breakdown: F2 (snoop) - The Syscall Tracer

This is how `strace` works under the hood. You will use `ptrace`, the API that allows one process to control the execution of another and inspect its CPU registers.

Step 1: Data Structures

You need a way to track the statistics of syscalls. Since Linux has around 300+ syscalls, a static array indexed by the syscall number is perfect.

```
typedef struct {
    int syscall_number;
```

```

    const char *name; // e.g., "read", "write". Look up a syscall table!
    int count;
    double total_time_seconds;
    int first_occurrence; // To break sorting ties
} SyscallStat;

SyscallStat stats[MAX_SYSCALLS];
// Initialize this array once with zeroes and the names of known syscalls.

```

Step 2: Attachment / Initialization

You have two modes: launching a new command, or attaching to an existing one.

- **Mode A (New Command):**

- `fork()` a child.
- *Child:* Call `ptrace(PTRACE_TRACEME, 0, NULL, NULL)`, then `execvp()`.
- *Parent:* Wait for the child to stop.

- **Mode B (-p pid):**

- Call `ptrace(PTRACE_ATTACH, pid, NULL, NULL)`.
- Wait for the process to stop.

Step 3: The Tracing Loop (The State Machine)

`ptrace` doesn't just stop the process *once* per syscall; it stops it **twice**: once right before the kernel executes the syscall (Entry), and once right after it finishes (Exit). You must track this state!

```

bool in_syscall_entry = true;
struct timespec start_time, end_time;
int current_syscall = -1;

WHILE true:
    // Tell the child to run until the next syscall entry/exit
    ptrace(PTRACE_SYSCALL, child_pid, NULL, NULL);
    waitpid(child_pid, &status, 0);

    IF WIFEXITED(status) OR WIFSIGNALED(status):
        BREAK // The traced process died. Trace is over.

    IF in_syscall_entry:
        // 1. We are entering a syscall.
        // 2. Read the CPU registers to find the syscall number.
        //    (On x86_64, the syscall number is in the orig_rax register).
        ptrace(PTRACE_GETREGS, child_pid, NULL, &regs);
        current_syscall = regs.orig_rax;

        // 3. Record the start time.
        clock_gettime(CLOCK_MONOTONIC, &start_time);

        in_syscall_entry = false; // Next stop will be the exit

    ELSE:
        // 1. We are exiting the syscall.
        // 2. Record the end time.
        clock_gettime(CLOCK_MONOTONIC, &end_time);

        // 3. Calculate elapsed time and update your stats array!
        Update stats[current_syscall].count++;
        Update stats[current_syscall].total_time_seconds += (end - start);

        in_syscall_entry = true; // Next stop will be an entry

```

Step 4: Sorting and Reporting

Once the loop breaks, you must package the active entries in your stats array into a temporary array, sort them using `qsort()` (Custom

comparator: sort by count descending, then by `first_occurrence` ascending), and print the formatted table. If a syscall number has no name in your lookup table, print `syscall_%d`.

4. Edge Cases & Pitfalls to Watch Out For

1. **readlink Does Not Null-Terminate!** Unlike string functions, `readlink` returns the number of bytes written to the buffer, but it *does not* append a `\0` character. You must manually do this: `ssize_t len = readlink(path, buf, sizeof(buf) - 1); if (len != -1) buf[len] = '\0';` If you forget this, `spy` will print trailing garbage memory.
2. **ptrace Architecture Dependence:** Getting the syscall number relies on reading CPU registers (`struct user_regs_struct`). If you are compiling on a 64-bit x86 Linux machine, the syscall number is in `orig_rax`. If you are on an ARM machine (like an M1/M2 Mac running a Linux VM, or a Raspberry Pi), the registers are named completely differently (e.g., `regs[7]` or `x8`). Ensure you know your target architecture.
3. **Hanging Attach (snoop -p):** If you `PTRACE_ATTACH` to a process that is currently asleep (e.g., you are snooping a `sleep 100` command), the `waitpid` might block for a moment, or the process might not hit a syscall exit immediately. `PTRACE_ATTACH` sends a `SIGSTOP`. Make sure you `waitpid` immediately after attaching to catch that `SIGSTOP` before beginning the `PTRACE_SYSCALL` loop.
4. **Zombie Tracees:** If your shell crashes while snooping, the traced process will be left in a stopped state forever. For robust cleanup, implement a signal handler in your shell that catches `SIGINT` (Ctrl+C), calls `ptrace(PTRACE_DETACH, child_pid, NULL, NULL)` to release the captive process, and *then* exits.

User prompt: *remaining part*

Response: Welcome to Kernel Space! You have officially left the safety of user-level C programming. You are no longer interacting with the OS; **you are the OS**.

Implementing a Multi-Level Feedback Queue (MLFQ) scheduler in xv6 requires modifying the core kernel loop, intercepting hardware timer interrupts, and managing process states directly.

Here is your architectural roadmap for building the MLFQ scheduler in xv6.

1. Phase 1: The Build System (Makefile)

The spec requires that you choose the scheduler at compile time using a macro.

Architectural Logic: In the xv6 `Makefile`, look for where `CFLAGS` is defined. You need to conditionally append a macro definition based on the user's input.

- If `SCHEDULER` is passed as an argument (e.g., `make qemu SCHEDULER=MLFQ`), you pass `-DMLFQ` to `gcc`.
- If it is not passed, default to `-DRR` (Round Robin).

In your C code (like `trap.c` and `proc.c`), you will use preprocessor directives (`#ifdef MLFQ ... #else ... #endif`) to separate your new logic from the original xv6 logic.

2. Phase 2: Kernel Data Structures (proc.h)

The original xv6 `struct proc` only tracks basic things like process state, page directory, and PID. To support MLFQ and gather metrics for your report, you must expand its memory footprint.

Data Structure Upgrades: Add these conceptual fields to `struct proc`:

1. **Queue Tracking:** `int priority` (0 to 3) and `int ticks_consumed` (how many ticks it has spent running in its *current* time slice).
2. **Queue Linking:** xv6's default scheduler just loops through a fixed array of 64 processes looking for `RUNNABLE`. For MLFQ, **order matters** (e.g., "inserted at the tail"). The easiest way to maintain queues in C without `malloc` is to add a `struct proc *next` pointer directly into the `struct proc` to form a Linked List.
3. **Metrics (For the Report):**
 - `creation_time` (recorded in `allocproc`).
 - `first_run_time` (recorded the first time it gets the CPU).
 - `end_time` (recorded in `exit`).

3. Phase 3: The Queuing Architecture (proc.c)

Since you can't use dynamic memory in the xv6 kernel, you need static queue heads and tails.

Architectural Logic: Define global arrays for your queue pointers:

```
struct proc *mlfq_head[4];
struct proc *mlfq_tail[4];
```

Write three helper functions to manage them safely:

1. `enqueue(struct proc *p, int priority)`: Adds `p` to the tail of `mlfq_tail[priority]`.
2. `dequeue(int priority)`: Removes and returns the process at `mlfq_head[priority]`.
3. `remove_process(struct proc *p)`: If a process is killed or goes to sleep while in the queue, you must be able to remove it from the middle of a linked list.

4. Phase 4: The Scheduler Loop (scheduler() in proc.c)

The `scheduler()` function is an infinite loop that runs on the CPU when no user process is running. You must wrap the original xv6 for loop in `#ifndef MLFQ` and write your own for `#ifdef MLFQ`.

Step-by-Step Flow:

1. Loop from Priority 0 down to 3.
2. If `mlfq_head[priority]` is not empty:
 - Dequeue the process.
 - Acquire the process lock (`acquire(&ptable.lock)` in standard xv6).
 - Update its state to `RUNNING`.
 - **Context Switch:** Call `swtch()` to hand the CPU over to this process.
 - *(The kernel stops executing here for this process. When `swtch()` returns, the process has yielded or been preempted).*
 - Upon return, check the process state. If it is still `RUNNABLE` (meaning it was preempted by a timer, not put to sleep by I/O), you must re-enqueue it. **But to which queue?** It depends on if it used up its time slice (handled in Phase 5).
3. If all queues are empty, the CPU should halt/wait for an interrupt (this logic already exists in xv6).

5. Phase 5: Timer Interrupts & Preemption (trap.c)

The hardware timer fires a `T_IRQ0 + IRQ_TIMER` interrupt periodically. This is the heartbeat of your MLFQ.

Architectural Logic (Inside the timer trap case):

1. **Global Tick Update:** Increment the global `ticks` variable.
2. **The 48-Tick Boost:** Check if `ticks % 48 == 0`.
 - If yes, loop through every single process in the system.
 - If it is `RUNNABLE` or `RUNNING`, reset its `priority = 0` and `ticks_consumed = 0`.
 - Rebuild your queue linked lists so everything is in Queue 0.
3. **Time Slice Tracking:** If there is a `RUNNING` process (i.e., `myproc() != 0`):
 - Increment its `ticks_consumed`.
 - Determine its allowed limit based on its priority (1, 4, 8, or 16).
 - If `ticks_consumed >= limit`:
 - Demote the priority (e.g., if `priority < 3`, `priority++`).

- Reset `ticks_consumed = 0`.
- Call `yield()` to force it to give up the CPU.

6. Phase 6: Lifecycle Hooks (Process Creation, I/O, Exit)

You must inject MLFQ logic into the process lifecycle functions in `proc.c`.

1. **`allocproc()`**: When a process is born, set `priority = 0`, `ticks_consumed = 0`, and enqueue it to Queue 0.
2. **`sleep()` / `wait()`**: When a process voluntarily gives up the CPU to wait for I/O, it calls `sleep()`. It leaves the `RUNNABLE` state. You don't need to demote it.
3. **`wakeup()`**: When I/O finishes, `wakeup()` marks a process as `RUNNABLE` again.
 - **Rule 5 Check**: The spec states: *"inserted at the tail of the same queue... priority is unchanged."*
 - You must enqueue(`p`, `p->priority`) right here inside `wakeup()`.
4. **`exit()`**: Make sure to clean up your metric timestamps and ensure it is removed from any queues.

7. Phase 7: Generating Metrics & The User-Space Test

To satisfy the report requirements (Turnaround Time, Response Time, Waiting Time), your OS needs to export the timestamps you collected.

Architectural Logic:

- **Response Time**: `first_run_time - creation_time`
- **Turnaround Time**: `end_time - creation_time`
- **Wait Time**: Turnaround Time - Total time spent actually `RUNNING`.

Since xv6 doesn't have a way to return this data to the user easily, the standard approach is to modify the `wait()` system call (or create a new `waitx()` syscall) so that when a parent waits for a child to die, the kernel passes these metrics back to the parent via pointer arguments.

`schedulertest.c`: Write a user-space C program that forks multiple children.

- **CPU-Bound Child**: Just does an infinite math loop (`for(int i=0; i<100000000; i++) x = x + 1;`).
- **I/O-Bound Child**: Loops calling `sleep(10)` or reading from a file, simulating giving up the CPU. This diverse workload will prove your MLFQ works: the CPU-bound processes will rapidly sink to Queue 3, while the I/O-bound processes will voluntarily yield and remain in Queue 0 or 1.

8. Edge Cases & Pitfalls to Watch Out For

1. **The Lock Trap**: Whenever you modify queue pointers, you must hold the `ptable.lock`. If an interrupt fires and calls `wakeup()` while `scheduler()` is modifying the linked list without a lock, your OS will instantly panic or corrupt memory.
2. **The "Preempt from Above" Rule**: Rule 2 states you *always* run the highest priority queue. If a process is running in Queue 2, and a keyboard interrupt wakes up an I/O process in Queue 0, the Queue 2 process must be preempted immediately at the next tick. Your `trap.c` logic must check if there are processes in higher queues than the currently running process, and if so, call `yield()`.
3. **Boost List Corruption**: When the 48-tick boost happens, you cannot just blindly change priorities. You must completely empty Queues 1, 2, and 3, and append all those processes to the tail of Queue 0, ensuring your linked list next pointers are perfectly updated. If you drop a pointer here, the process is lost forever (Memory Leak in Kernel Space!).

User prompt: prepare detailed test cases for both shell and xv6

Response: As someone who evaluates algorithmic logic and edge cases for Data Structures and CPro, you know the difference between code that works on the "happy path" and code that survives a rigorous auto-grader. To ensure your shell and kernel modifications are bulletproof, you must test them against adversarial inputs, resource exhaustion, and concurrency race conditions.

Here is a comprehensive, auto-grader-style test suite designed to break your implementations.

I. C-Shell Test Suite

Run these tests manually or script them. Keep an eye on `htop` or `ps -a` in another terminal window to watch for zombie processes or file descriptor leaks while you test.

Part A: Lexing & Parsing (The Automata Breakers)

Scenario	Input	Expected Output / Behavior	The "Gotcha" Tested
Maximal Munch	cat <in.txt>out.txt>>app.txt	Executes perfectly. 3 file redirections.	Lexer must separate operators from words even without spaces.
Quote Masking	`echo "cat < a.txt	sort``	Prints exactly: cat < a.txt sort
Empty Strings	echo "" "" ""	Prints three empty strings (just spaces and a newline).	Lexer must yield a valid token for closed, empty quotes, not ignore them.
Unclosed Quotes	echo "hello world	cshell: invalid syntax	Must gracefully reject and return to prompt without leaking memory.
Trailing Escape	cat my\ file\	cshell: invalid syntax	A backslash at the absolute end of the input stream.

Part B: Shell Intrinsic (State & Metadata)

Scenario	Input / Action	Expected Output / Behavior	The "Gotcha" Tested
Frecency Cascade	1. hop /tmp	Hops to /tmp.	Frecency substring matching and score sorting.
	2. hop /usr/bin		
	3. hop tm		
Frecency Phantom	rm -rf dir1, then hop dir1	Skips dir1 and finds the next best match.	hop must check access () before jumping, handling deleted directories gracefully.
Reveal Recursion	reveal -t in a huge directory.	Prints tree, doesn't crash.	If you forget to skip . and . . in your recursive reveal, you will trigger an infinite loop and stack overflow.
Peek Boundary	peek -r on a 4097-byte file.	Reverses perfectly, no lost characters.	Tests your chunk-reading math. If your buffer is 4096 bytes, does the chunk boundary tear a word in half?
Locate Duplicates	locate bash	Prints exact absolute paths.	If the current directory is also in \$PATH, does it print the same file twice? (It shouldn't).

Part C: Execution, Pipes & Redirection (The Deadlock Zone)

Scenario	Input / Action	Expected Output / Behavior	The "Gotcha" Tested
Dangling Pipe	cat /dev/urandom head -n 5	Prints 5 lines and returns to prompt.	If the parent shell doesn't close the write-end of the pipe, head exits, but cat runs forever.
Redirection Override	echo "fail" > out.txt echo "pass" > out.txt	out.txt contains only "pass".	File redirections must securely overwrite the pipe wiring.

Scenario	Input / Action	Expected Output / Behavior	The "Gotcha" Tested
Multi-Input Merge	cat < file1.txt < file2.txt	Outputs contents of file1 then file2 continuously.	Requires a background helper process to merge streams without blocking the parent shell.
Pipeline Failure	echo hi dosakdaoskdos wc -l	Prints cshell: command not found (dosakdaoskdos), then prints 0.	The failure of a middle command must not crash the rest of the pipeline.
FD Exhaustion	Run ls sort 500 times in a row.	Should not crash with "Too many open files".	Parent must close() pipe file descriptors after every loop iteration.

Part D-F: Concurrency & Signals

Scenario	Input / Action	Expected Output / Behavior	The "Gotcha" Tested
Zombie Check	sleep 2 &, wait 3s, run activities	Process is gone from activities list.	Your SIGCHLD handler must reap the process using waitpid(..., WNOHANG).
Signal Race	Run sleep 10, hit Ctrl+C.	sleep dies, shell prompt reappears. Shell does NOT exit.	Shell must ignore SIGINT, but the foreground process group must receive it.
Pipeline PGID	sleep 50 sleep 60 &, run activities	Both show the SAME PGID.	setpgid must be applied correctly to every child in a pipeline.
Snoop Attach	sleep 100 &, then snoop -p <pid>	Attaches, waits, prints stats when killed.	Tests PTRACE_ATTACH and handling of processes that are asleep.

II. xv6 MLFQ Scheduler Test Suite

In kernel space, you cannot rely on `printf` for standard testing. Your primary debugging tool is `procdump()` triggered by pressing `Ctrl+P` in the QEMU terminal.

Write a user-space C program (`schedulertest.c`) that forks specific workloads to verify these transitions.

1. The Preemption Test (Rule 2)

- **Setup:** Modify your test program to launch a CPU-bound process that runs a heavy for loop. Wait a few seconds, then fork a new process.
- **Action:** Press `Ctrl+P`.
- **Expected:** The first process should be in Queue 3 (lowest). The new process starts in Queue 0. The OS must immediately preempt the Queue 3 process and run the Queue 0 process.
- **Failure State:** If the new process waits for the old process to finish its long time slice, your `trap.c` logic isn't preempting correctly at tick boundaries.

2. The Time Slice Exhaustion Test (Rule 4)

- **Setup:** Fork a single CPU-bound process.
- **Action:** Spam `Ctrl+P` repeatedly as fast as you can.
- **Expected:** You should see its priority drop: `0 -> 1 -> 2 -> 3`. You should also see its `ticks_consumed` counter reset to 0 at each demotion.
- **Failure State:** The process stays in Queue 0 forever, or it drops to Queue 4 (which shouldn't exist).

3. The I/O Voluntarism Test (Rule 5)

- **Setup:** Fork a process that loops `sleep(1)`.
- **Action:** Press `Ctrl+P` occasionally.

- **Expected:** The process should always remain in Queue 0 (or Queue 1). It yields before hitting the time slice limit (1 tick).
- **Failure State:** The process sinks to Queue 3. This means your `wakeup()` function isn't restoring it to its original queue, or you are improperly incrementing `ticks_consumed` while it sleeps.

4. The Anti-Starvation Boost Test (Rule 7)

- **Setup:** Fork three CPU-bound processes. Let them all sink to Queue 3.
- **Action:** Watch the global `ticks` variable (you can print it during `procdump`). Wait for it to hit a multiple of 48.
- **Expected:** Instantly, all three processes should jump back to Queue 0.
- **Failure State:** The linked list pointers for Queue 0 get corrupted (e.g., the tail pointer points to NULL, or processes get lost in the void and never scheduled again).

User prompt: *i need a script for making plot*

Response: Here is a Python script using `matplotlib` to generate the exact queue vs. time scatter plot required for your MLFQ report.

To make this work, the script expects a simple comma-separated log file (`mlfq_log.txt`). It is fully configured to map the PIDs to distinct colors, lock the Y-axis to your 4 queues, and automatically draw red dashed lines every 48 ticks to visually highlight your anti-starvation priority boosts.

The Python Script (`plot_mlfq.py`)

```
import matplotlib.pyplot as plt

# Configuration
DATA_FILE = 'mlfq_log.txt'
BOOST_INTERVAL = 48

ticks = []
pids = []
queues = []

# 1. Parse the data
print(f"Reading data from {DATA_FILE}...")
with open(DATA_FILE, 'r') as f:
    for line in f:
        line = line.strip()
        # Skip empty lines or headers
        if not line or line.startswith('#'):
            continue
        try:
            # Expected format: tick,pid,queue
            t, p, q = map(int, line.split(','))
            ticks.append(t)
            pids.append(p)
            queues.append(q)
        except ValueError:
            pass

if not ticks:
    print("No valid data found! Ensure your file format is: tick,pid,queue")
    exit()

# 2. Create the scatter plot
plt.figure(figsize=(12, 6))
# tab10 provides up to 10 distinct colors for different PIDs
scatter = plt.scatter(ticks, queues, c=pids, cmap='tab10', alpha=0.9, edgecolors='black', s=60)

# 3. Formatting
plt.title('MLFQ Scheduler Timeline', fontsize=14, fontweight='bold')
plt.xlabel('Time Elapsed (Ticks)', fontsize=12)
plt.ylabel('Queue ID (0 = Highest, 3 = Lowest)', fontsize=12)

# Lock Y-axis strictly to 0, 1, 2, 3 and invert so Queue 0 is at the top visually
```

```

plt.yticks([0, 1, 2, 3])
plt.gca().invert_yaxis()

# 4. Highlight the 48-tick priority boosts
max_tick = max(ticks)
for boost_tick in range(BOOST_INTERVAL, max_tick + 1, BOOST_INTERVAL):
    # Draw a vertical line for every boost boundary
    plt.axvline(x=boost_tick, color='red', linestyle='--', alpha=0.5)

# Add a label for the first boost line so it appears in the legend
if max_tick >= BOOST_INTERVAL:
    plt.axvline(x=BOOST_INTERVAL, color='red', linestyle='--', alpha=0.5, label='Priority Boost (48 ticks)')

# 5. Legend for PIDs
handles, labels = scatter.legend_elements()
pid_legend = plt.legend(handles, labels, title="Process ID", loc="lower right")
plt.gca().add_artist(pid_legend)

# Add the boost line to the legend if applicable
plt.legend(loc="upper right")

plt.grid(True, axis='y', linestyle=':', alpha=0.7)
plt.tight_layout()

# Save the high-res image for the report
plt.savefig('mlfq_timeline.png', dpi=300)
print("Plot saved as mlfq_timeline.png")
plt.show()

```

How to integrate this with xv6

To get the data out of your kernel and into this script, you need to print a formatted string every time a process runs.

1 Add the print statement to xv6

Inside `proc.c`, specifically in your `scheduler()` loop where you actually context switch to a process, add a `cprintf` that outputs the global tick, the PID, and the current queue.

```
cprintf("%d,%d,%d\n", ticks, p->pid, p->priority);
```

2 Run your test workload

Boot xv6 using `make qemu SCHEDULER=MLFQ` and run your `schedulertest` program.

3 Save the output

Copy the comma-separated output printed in your terminal and paste it into a file named `mlfq_log.txt` in the same directory as the Python script.

4 Generate the graph

Run `python3 plot_mlfq.py`. It will immediately output `mlfq_timeline.png` which you can embed directly into your report markdown or PDF.